

Minor Project Report on

Identification of Encryption Algorithms and Plaintext Recovery Using Machine Learning

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY (MINOR)

in

MACHINE LEARNING

by

Nithin S (221IT085)

under the guidance of

Dr. Purushothama B R



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE - 575025

November, 2025

DECLARATION

by the B.Tech Student

We hereby *declare* that the Minor Project Work Report entitled “***Identification of Encryption Algorithms and Plaintext Recovery Using Machine Learning***” , which is being submitted to the **National Institute of Technology Karnataka, Surathkal**, for the award of the Degree of Bachelor of Technology (Minor) in Machine Learning, is a *bonafide report of the work carried out by us*. The material contained in this Major Project Report has not been submitted to any University or Institution for the award of any degree.

Name of the Student (Registration Number) with Signature

(1) Nithin S (221IT085)

Department of Information Technology

Place : NITK, Surathkal

Date : 10/11/2025

CERTIFICATE

This is to *certify* that the Minor Project Work Report entitled “***Identification of Encryption Algorithms and Plaintext Recovery Using Machine Learning***” submitted by

Name of the Student (Registration Number)

(1) Nithin S (221IT085)

as the record of the work carried out by them, is *accepted as the B.Tech. Minor Project work report submission* in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology (Minor) in Machine Learning in the Department of Information Technology, NITK Surathkal.

Guide Signature with date

Dr. Purushothama B R

Associate Professor

Department of Information Technology

NITK Surathkal-575025

Chairman - DUGC

Signature With Date and Seal

ABSTRACT

The automated identification of cryptographic algorithms from ciphertext represents a fundamental challenge in modern cryptanalysis, security assessment, and forensic investigation. This research presents a comprehensive investigation into ciphertext-based encryption algorithm classification using progressively sophisticated machine learning and deep learning methodologies. The study systematically explores six distinct approaches, advancing from traditional statistical feature extraction to state-of-the-art recurrent neural architectures. Initially, we replicate and extend the randomness-based feature extraction framework utilizing NIST statistical tests combined with ensemble learning methods including Random Forest, Logistic Regression, and Hybrid Random Forest-Logistic Regression models, establishing baseline performance benchmarks with accuracies reaching 38% for five-class classification tasks involving Advanced Encryption Standard, Triple Data Encryption Standard, Blowfish, CAST, and RC2. Building upon these foundations, we transition to deep learning paradigms where Long Short-Term Memory networks are employed to model temporal dependencies inherent in ciphertext sequences, achieving 52.3% accuracy on multi-algorithm classification. Bidirectional LSTM architectures demonstrate superior performance at 58.7% by simultaneously capturing forward and backward contextual patterns in encrypted data. Gated Recurrent Unit models provide computational efficiency with 54.1% accuracy, while Bidirectional GRU variants reach 61.2%, establishing recurrent architectures as more effective than traditional feature-based methods. A novel contribution of this work is the introduction of round-based cryptanalysis, wherein classification is performed on intermediate encryption states rather than exclusively on final ciphertexts. This approach reveals a critical insight: early-round ciphertexts from rounds 1-3 exhibit substantially higher algorithmic discriminability with accuracies exceeding 85-97%, while classification performance systematically degrades through progressive rounds, approaching random-guess baseline of 50% at full encryption rounds 10-16. Comprehensive pairwise experiments across algorithm combinations including AES, DES, IDEA, Camellia, ChaCha20, and RC4 demonstrate round 1 accuracies ranging from 96.8% to 99.0%, degrading to 69.6-73.8% at final rounds. These empirical findings confirm theoretical expectations regarding cryptographic diffusion properties while simultaneously establishing that partial-round analysis provides actionable intelligence in real-world cryptanalytic scenarios. In addition to algorithm identification, this study further investigates the feasibility of leveraging

learned representations to approximate partial plaintext recovery under constrained settings, revealing preliminary insights into the boundaries of ciphertext-only inference. The systematic performance degradation across encryption rounds serves as empirical validation of the security properties of modern cryptographic primitives, demonstrating that well-designed ciphers successfully eliminate algorithmic signatures through successive transformations. This work establishes comprehensive benchmarks for neural-network-based cryptographic algorithm identification, provides practical insights for security assessment methodologies, and demonstrates both the capabilities and fundamental limitations of machine learning approaches when confronted with cryptographically secure transformations.

Keywords: Cryptographic Algorithm Identification, Deep Learning, LSTM, Bidirectional Recurrent Networks, GRU, Round-Based Cryptanalysis, Ciphertext Classification, Ensemble Learning, NIST Randomness Tests, Neural Cryptanalysis, Plaintext Recovery

CONTENTS

LIST OF FIGURES	vii
LIST OF TABLES	viii
1 INTRODUCTION	1
1.1 Overview	1
1.2 Motivation	2
1.3 Problem Statement	2
1.3.1 Research Questions	3
2 Preliminaries	4
2.0.1 Cryptographic Fundamentals	4
2.0.2 Cryptographic Algorithm Identification Problem	6
2.0.3 Feature Extraction from Ciphertext	7
2.0.4 Machine Learning Classification Techniques	9
2.0.5 Hierarchical Classification Strategy	10
2.0.6 Evaluation Metrics	11
2.0.7 Round-Based Security Analysis	12
2.0.8 Dataset Construction	13
2.0.9 Implementation Details	14
3 LITERATURE REVIEW	16
3.1 Related Works	16
3.2 Outcome of the Literature Survey	21
3.3 Problem Statement	22
3.4 Objectives of the Project	23
4 Methodology	25
4.1 Dataset Preparation	25
4.1.1 Generation Protocol	25
4.2 Feature Extraction Methodologies	26
4.2.1 NIST Statistical Test Suite Features	26
4.2.2 Entropy-Based Features	31

4.2.3	Byte-Level Statistical Features	31
4.2.4	Bit-Level Structural Features	31
4.2.5	Spectral Features	32
4.2.6	Feature Fusion and Normalization	32
4.2.7	NIST Feature Selection via DRRA	32
4.3	Deep Learning Models for Cryptographic Analysis	34
4.3.1	Computational Infrastructure	35
4.3.2	Encryption Scheme Identification	35
4.3.3	Deception Detection in Encrypted Messages	36
4.3.4	Plaintext Recovery via Seq2Seq LSTM	37
4.3.5	Vigenère Cipher Key Length Prediction	38
5	Experimental Framework, Results and Analysis	40
5.1	Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Seven Machine Learning Models	40
5.1.1	Problem Formulation	40
5.1.2	Dataset Configuration	42
5.1.3	Feature Engineering Pipeline	42
5.1.4	Classification Models	43
5.1.5	Experimental Design	44
5.1.6	Results	45
5.1.7	Binary Classification: Pairwise Algorithm Identification	45
5.1.8	Algorithm Pair Similarity Analysis	47
5.1.9	Three-Class Classification	48
5.1.10	Confusion Matrix Analysis: Three-Class Case	49
5.1.11	Six-Class Classification	50
5.1.12	Per-Cipher Performance in Six-Class Setting	52
5.1.13	Feature Importance Analysis	53
5.1.14	Model Computational Complexity Analysis	55
5.1.15	Cross-Validation Stability Analysis	56
5.1.16	Discussion	57
5.1.17	Ciphertext-Only Identification Challenges	57
5.1.18	Stream vs Block Cipher Dichotomy	58

5.1.19	Ensemble Learning Advantages	59
5.1.20	Practical Implications	59
5.1.21	Comparison with Random Baselines	60
5.1.22	Conclusion	61
5.2	Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Multi-Layer Perceptron	62
5.2.1	Problem Formulation and Architecture	62
5.2.2	Feature Engineering Pipeline	62
5.2.3	Neural Network Architecture	64
5.2.4	Baseline Comparison Models	65
5.2.5	Experimental Design	65
5.2.6	Results	66
5.2.7	Training and Implementation Details	71
5.2.8	Computational Complexity Comparison	71
5.2.9	Cross-Validation Stability Analysis	71
5.2.10	Discussion	73
5.2.11	Conclusion	75
5.3	Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Recurrent Neural Networks	77
5.3.1	Problem Formulation and Architecture	77
5.3.2	Feature Engineering Pipeline	78
5.3.3	Neural Network Architectures	78
5.3.4	Experimental Design	80
5.3.5	Plaintext Reconstruction Task	81
5.3.6	Results	81
5.3.7	Plaintext Reconstruction Evaluation	88
5.3.8	Computational Complexity Analysis	90
5.3.9	Cross-Validation Stability Analysis	91
5.3.10	Discussion and Key Findings	93
5.3.11	Conclusion	93
5.4	Experimental Framework for Round-Based Metrics	93
5.4.1	2-Class Binary Classification	94

5.4.2	3-Class Multi-Cipher Classification	96
5.4.3	6-Class Classification (RC4 Withheld)	96
5.4.4	Interpretation	97
6	Conclusion	98
	REFERENCES	99

LIST OF FIGURES

4.1	Feature Selection Workflow	33
4.2	Seq2Seq Architecture with Attention Mechanism	37
5.1	Flow chart of block cipher algorithm identification scheme based on HRFLR	41
5.2	Flow chart for identifying block cipher algorithm using the MLP method	63
5.3	RNN pipeline for ciphertext-only cipher identification	77
5.4	Round-based cryptographic leakage evaluation pipeline	94

LIST OF TABLES

2.1	Round Structure of Cryptographic Algorithms	6
4.1	Dataset Structure Overview	26
4.2	NIST Statistical Test Suite Overview	27
4.3	Top 10 Discriminative NIST Features (DRRA Selected)	34
4.4	Neural Network Architectures for Cryptographic Tasks	34
4.5	Optimal Hyperparameters for CNN and LSTM	36
4.6	Optimal Hyperparameters for BiLSTM and BiGRU	36
4.7	Model Complexity Comparison	37
5.1	Dataset Configuration for TLS Algorithm Identification	42
5.2	Binary Classification Results: Model Comparison Across Key Cipher Pairs	45
5.3	Binary Classification: Aggregate Model Performance (Mean Across 21 Pairs)	46
5.4	Algorithm Pair Similarity Matrix Using HRFLR (Lower accuracy = Higher similarity)	47
5.5	Three-Class Classification: Model Comparison (Representative Triplets)	48
5.6	Three-Class Classification: Aggregate Model Performance	48
5.7	Confusion Matrix: RC4, AES, ChaCha20 (Three-Class, HRFLR) . . .	49
5.8	Six-Class Classification: Comprehensive Model Comparison	50
5.9	Per-Cipher F1-Scores: Six-Class Classification (All Models)	52
5.10	Top 10 Discriminative Features (Random Forest Importance)	54
5.11	Computational Complexity: Training and Inference Time	55
5.12	5-Fold Cross-Validation: Model Stability (Six-Class)	56
5.13	Comparison of model performance against random and majority-class baselines	60
5.14	Binary Classification Results: MLP Performance Across All Cipher Pairs	66
5.15	Binary Classification: MLP vs Baseline Models (Mean Across 21 Pairs)	66
5.16	Three-Class Classification: MLP Performance (Representative Triplets)	68
5.17	Three-Class Classification: MLP vs Baseline Models (Mean Across 35 Triplets)	68

5.18 Six-Class Classification: MLP Performance Metrics	69
5.19 Six-Class Classification: Comprehensive Model Comparison	70
5.20 MLP Training Configuration and Computational Requirements	72
5.21 Computational Complexity: MLP vs Traditional Models (Six-Class Task)	73
5.22 5-Fold Cross-Validation: MLP Stability Analysis (Six-Class)	73
5.23 Performance Comparison: MLP vs Best Traditional Model (HRLFR)	74
5.24 Binary Classification Results: RNN Model Comparison (Mean Across 21 Pairs)	82
5.25 Three-Class Classification: RNN Model Comparison (Average Across 35 Triplets)	84
5.26 Six-Class Classification: RNN Model Comparison	85
5.27 Six-Class Classification: Comprehensive Model Comparison	87
5.28 Plaintext Reconstruction: Sequence-to-Sequence LSTM Performance	88
5.29 Computational Complexity: RNN Model Comparison (Six-Class Task)	90
5.30 5-Fold Cross-Validation: RNN Stability Analysis (Six-Class)	91
5.31 HRLFR binary classification accuracy (%) across rounds	95
5.32 MLP binary classification accuracy (%) across rounds	95
5.33 BiLSTM binary classification accuracy (%) across rounds	96
5.34 3-class classification: averaged accuracy (%) over 35 triplets	96
5.35 6-class classification accuracy (%) with RC4 withheld	97

CHAPTER 1

INTRODUCTION

Cryptanalysis focuses on understanding and breaking encryption schemes without prior knowledge of the key. While traditional approaches rely on mathematical and statistical techniques, the increasing complexity of modern cryptography has made many classical methods less effective for real-world systems.

Encrypted communication now underpins global digital infrastructure across banking, communication networks, cloud services, and government systems. In many operational scenarios, only ciphertext is visible, and identifying the encryption algorithm becomes a critical first step in security auditing, digital forensics, and threat analysis.

Recent advances in machine learning and deep learning have introduced new opportunities for analyzing encrypted data. Rather than relying on handcrafted heuristics, data-driven models learn subtle patterns from ciphertext, enabling automated algorithm identification and cryptographic behavior analysis. This makes them promising tools for evaluating encryption systems and understanding how cipher structures manifest in encrypted output.

This work investigates machine learning and deep learning for ciphertext-based algorithm identification, progressing from statistical features with classical classifiers to recurrent neural networks that learn directly from raw ciphertext sequences. We also introduce round-based cryptanalysis to analyze how algorithm distinguishability changes across encryption rounds, revealing how rapidly modern ciphers achieve diffusion. In addition, we extend this study to explore whether learned ciphertext representations can support limited plaintext recovery in controlled environments, examining the feasibility and boundaries of ciphertext-only inference.

1.1 Overview

This research examines automated cryptographic algorithm identification using:

- Statistical feature extraction with ensemble learning
- Deep learning models applied directly to ciphertext (LSTM, BiLSTM, GRU,

BiGRU)

- Round-based analysis to study diffusion and algorithm signature disappearance
- Preliminary evaluation of plaintext recovery feasibility from ciphertext representations

The experiments consider block and stream ciphers including AES, DES, 3DES, IDEA, Camellia, Blowfish, CAST, RC2, RC4, and ChaCha20. Statistical methods provide baseline accuracy, while recurrent neural models improve performance by learning temporal patterns. Round-based results show high accuracy in early rounds and near-random behavior after full diffusion, consistent with cryptographic theory.

1.2 Motivation

Analysts frequently encounter encrypted data without knowing the encryption algorithm. Manual inspection is slow and expertise-dependent, while automated classification offers scalable and rapid assessment. Machine learning enables systems to detect algorithmic signatures in ciphertext and provides an empirical means to test cryptographic strength. Extending this analysis to exploratory plaintext recovery allows investigation into whether deep learning models can infer limited plaintext structure in constrained settings, highlighting practical security boundaries.

1.3 Problem Statement

Given ciphertext produced by an unknown encryption algorithm, the task is to classify the generating algorithm without plaintext or key information:

$$c = E_{a_i}(m, k) \Rightarrow a_i$$

An extended task examines algorithm identification from intermediate round outputs and explores whether limited plaintext recovery is possible from learned ciphertext features.

1.3.1 Research Questions

1. How effective are statistical features and ensemble models for ciphertext classification?
2. Do recurrent neural networks improve performance over handcrafted features?
3. Which model architecture yields the strongest results?
4. How does accuracy change across encryption rounds?
5. How does ciphertext size affect classification accuracy?
6. Can learned structures support any degree of plaintext recovery under constrained settings?
7. Do results align with cryptographic security expectations?

Summary

This chapter introduces the motivation and goals of applying machine learning to ciphertext analysis. The study benchmarks classical and deep learning approaches for algorithm identification and investigates algorithm detectability across encryption rounds. Results show strong early-round identification accuracy that declines as encryption progresses, confirming that modern ciphers converge to random-like output. The chapter also introduces exploratory plaintext recovery experiments, evaluating whether deep models can infer limited plaintext structure and demonstrating that modern encryption remains secure under realistic ciphertext-only conditions.

CHAPTER 2

Preliminaries

This section provides the foundational concepts and terminology necessary for understanding our cryptographic algorithm identification scheme. We discuss the basics of cryptographic systems, the machine learning techniques employed, and the evaluation metrics used throughout this work.

2.0.1 Cryptographic Fundamentals

Symmetric vs. Asymmetric Cryptography

Cryptographic systems are broadly categorized into two classes based on their key structure:

[Symmetric Cryptosystem] A symmetric cryptosystem is a cryptographic scheme where the same key k is used for both encryption and decryption operations. For a plaintext m and ciphertext c , we have:

$$c = E_k(m) \tag{2.1}$$

$$m = D_k(c) \tag{2.2}$$

where E_k and D_k denote the encryption and decryption functions, respectively.

[Asymmetric Cryptosystem] An asymmetric cryptosystem employs a pair of mathematically related keys: a public key pk for encryption and a private key sk for decryption. For a plaintext m and ciphertext c :

$$c = E_{pk}(m) \tag{2.3}$$

$$m = D_{sk}(c) \tag{2.4}$$

Symmetric cryptosystems are further divided into two primary categories:

- **Stream Ciphers:** Process plaintext bit-by-bit or byte-by-byte, generating a pseudorandom keystream that is combined with the plaintext.

- **Block Ciphers:** Process plaintext in fixed-size blocks (typically 64 or 128 bits), applying complex transformations through multiple rounds.

Block Cipher Structure and Round Functions

Block ciphers form the core of modern symmetric cryptography. They transform plaintext blocks into ciphertext blocks through iterative application of a round function.

[Block Cipher] A block cipher is a deterministic algorithm that operates on fixed-length groups of bits, called blocks. Let $\mathcal{P}$ and $\mathcal{C}$ denote the plaintext and ciphertext spaces, respectively, and $\mathcal{K}$ denote the key space. A block cipher is a family of permutations:

$$E : \mathcal{K} \times \mathcal{P} \rightarrow \mathcal{C}$$

such that for each key $k \in \mathcal{K}$, the function $E_k(\cdot) = E(k, \cdot)$ is a bijection from $\mathcal{P}$ to $\mathcal{C}$.

[Round Function] A round function F_r represents a single iteration of the cryptographic transformation within a block cipher. For round r with round key k_r , state s_r , the round function is:

$$s_{r+1} = F_r(s_r, k_r)$$

The complete encryption process applies R rounds:

$$c = F_R(F_{R-1}(\cdots F_1(m, k_1) \cdots, k_{R-1}), k_R)$$

The number of rounds varies across different cryptographic algorithms and directly impacts their security and computational complexity. Table 2.1 summarizes the round structure of algorithms studied in this work.

Modes of Operation

Block ciphers require modes of operation to process messages longer than a single block. The five standard modes are:

- **Electronic Codebook (ECB):** Each block is encrypted independently: $c_i = E_k(m_i)$

Table 2.1: Round Structure of Cryptographic Algorithms

Algorithm	Total Rounds	Block Size	Structure Type
AES-128	10	128 bits	Substitution-Permutation Network
DES	16	64 bits	Feistel Network
IDEA	8	64 bits	Modified Feistel Network
Camellia	18	128 bits	Feistel Network
ChaCha20	20	512 bits	ARX Construction
RC4	16*	Stream	Pseudo-rounds

RC4 is a stream cipher; rounds represent key scheduling iterations.

- **Cipher Block Chaining (CBC):** Each plaintext block is XORed with the previous ciphertext block: $c_i = E_k(m_i \oplus c_{i-1})$
- **Cipher Feedback (CFB):** Operates as a stream cipher using block cipher: $c_i = m_i \oplus E_k(c_{i-1})$
- **Output Feedback (OFB):** Generates keystream independently: $c_i = m_i \oplus o_i$ where $o_i = E_k(o_{i-1})$
- **Counter (CTR):** Encrypts counter values: $c_i = m_i \oplus E_k(\text{nonce} || \text{ctr}_i)$

2.0.2 Cryptographic Algorithm Identification Problem

[Cryptographic Algorithm Identification] Given a set of cryptographic algorithms $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$ and a ciphertext c generated by an unknown algorithm $a_i \in \mathcal{A}$, the cryptographic algorithm identification problem is to determine a mapping function:

$$\phi : \mathcal{C} \rightarrow \mathcal{A}$$

that correctly identifies $\phi(c) = a_i$ with maximum probability, where $\mathcal{C}$ denotes the space of all possible ciphertexts.

The identification task operates under the *ciphertext-only* scenario, where:

- The plaintext m is unknown
- The encryption key k is unknown

- Only the ciphertext c is available for analysis
- The algorithm belongs to a known set $\mathcal{A}$

2.0.3 Feature Extraction from Ciphertext

Randomness Testing Framework

The NIST Statistical Test Suite (SP 800-22) provides a comprehensive framework for evaluating the randomness of binary sequences. These tests are designed to detect deviations from true randomness that may indicate cryptographic weaknesses.

[Randomness Test] A randomness test is a statistical hypothesis test where:

- **Null Hypothesis (H_0):** The sequence is random
- **Alternative Hypothesis (H_1):** The sequence is non-random

Each test produces a p -value in the range $[0, 1]$, representing the probability that a perfectly random sequence would produce a test statistic at least as extreme as the observed value.

[p -value Interpretation] For a significance level α (typically $\alpha \in [0.001, 0.01]$):

- If $p \geq \alpha$: Accept H_0 (sequence appears random)
- If $p < \alpha$: Reject H_0 (sequence appears non-random)

The p -value itself serves as a continuous measure of randomness, where values closer to 1 indicate higher randomness and values closer to 0 indicate more structure.

Selected Randomness Tests

We employ the following NIST tests for feature extraction:

1. **Frequency (Monobit) Test:** Determines whether the number of ones and zeros in a sequence are approximately equal.
2. **Runs Test:** Examines the oscillation between zeros and ones, detecting rapid changes.

3. **Longest Run of Ones Test:** Checks whether the length of the longest run of ones is consistent with randomness.
4. **Binary Matrix Rank Test:** Checks for linear dependence among fixed-length substrings.
5. **Non-overlapping Template Matching Test:** Detects occurrences of pre-specified patterns.
6. **Maurer’s Universal Statistical Test:** Detects whether the sequence can be significantly compressed.
7. **Serial Test:** Examines the frequency of all possible overlapping m -bit patterns.
8. **Approximate Entropy Test:** Compares the frequency of overlapping blocks of consecutive lengths.
9. **Cumulative Sums Test:** Determines whether the cumulative sum of partial sequences is too large or too small.
10. **Random Excursions Test:** Examines the number of cycles having exactly K visits in a cumulative sum random walk.

[Feature Vector] Let $\mathcal{T} = \{t_1, t_2, \dots, t_m\}$ denote a set of m randomness tests. For a ciphertext c , the feature vector is defined as:

$$\mathbf{f}(c) = [p_1, p_2, \dots, p_m]^T$$

where p_i is the p -value returned by test t_i applied to ciphertext c .

Entropy-Based Features

In addition to NIST tests, entropy measures provide information about the randomness and information content of ciphertext:

[Shannon Entropy] For a discrete random variable X with alphabet $\mathcal{X}$ and probability mass function $p(x)$, the Shannon entropy is:

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

For ciphertext analysis, we compute byte-level and bit-level entropy as features.

2.0.4 Machine Learning Classification Techniques

Random Forest Classifier

[Random Forest] A Random Forest is an ensemble learning method that constructs N decision trees $\{T_1, T_2, \dots, T_N\}$ during training. Each tree T_i is trained on a bootstrap sample of the training data with a random subset of features. For classification, the final prediction is determined by majority voting:

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_N(\mathbf{x})\}$$

where $\mathbf{x}$ is the input feature vector.

Key properties of Random Forest:

- **Bootstrap Aggregating (Bagging):** Reduces variance by averaging predictions
- **Feature Randomness:** Decorrelates trees by using random feature subsets
- **Out-of-Bag (OOB) Estimation:** Provides unbiased error estimation

Logistic Regression

[Logistic Regression] Logistic Regression models the probability of class membership using the logistic function. For binary classification with feature vector $\mathbf{x} \in \mathbb{R}^d$ and parameters $\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

where $\sigma(\cdot)$ is the sigmoid function.

For multi-class classification with K classes, we use the softmax extension:

$$P(y = k|\mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

Hybrid Random Forest and Logistic Regression (HRFLR)

[HRFLR Model] The Hybrid Random Forest and Logistic Regression (HRFLR) model is a two-stage ensemble classifier:

1. **Stage 1 (Random Forest):** Train a Random Forest $\mathcal{F}$ on the original feature space $\mathbf{x} \in \mathbb{R}^d$
2. **Stage 2 (Feature Transformation):** Extract leaf node indices from each tree, creating augmented features $\mathbf{x}' \in \mathbb{R}^{d+N}$ where N is the number of trees
3. **Stage 3 (Logistic Regression):** Train a Logistic Regression classifier on the augmented feature space

The HRFLR model combines the strengths of both algorithms:

- Random Forest captures non-linear relationships and complex interactions
- Logistic Regression provides calibrated probability estimates and regularization
- The stacking approach reduces overfitting through cross-validation

2.0.5 Hierarchical Classification Strategy

[SSBA-Cluster Division] The Symmetric-Stream-Block-Asymmetric (SSBA) cluster division is a hierarchical taxonomy that organizes cryptographic algorithms into nested categories:

$$\mathcal{A} = \mathcal{C}_S \cup \mathcal{C}_A$$

where:

- $\mathcal{C}_S$: Symmetric cryptosystems, further divided as $\mathcal{C}_S = \mathcal{C}_{SS} \cup \mathcal{C}_{SB}$
 - $\mathcal{C}_{SS}$: Stream ciphers (e.g., RC4, ChaCha20, Salsa20)
 - $\mathcal{C}_{SB}$: Block ciphers (e.g., AES, DES, IDEA, Camellia)
- $\mathcal{C}_A$: Asymmetric cryptosystems (e.g., RSA, ECC, SM2)

The hierarchical classification proceeds through multiple layers:

1. **Layer 1:** Distinguish symmetric vs. asymmetric cryptosystems
2. **Layer 2:** Within symmetric systems, distinguish stream vs. block ciphers
3. **Layer 3:** Identify specific algorithm within each category

This approach reduces the complexity of multi-class classification by decomposing it into simpler binary or few-class problems at each level.

2.0.6 Evaluation Metrics

Confusion Matrix

[Confusion Matrix] For a classification problem with K classes, the confusion matrix $\mathbf{M} \in \mathbb{N}^{K \times K}$ is defined where element M_{ij} represents the number of instances of true class i predicted as class j .

For binary classification, we define:

- **True Positives (TP):** Correctly identified positive instances
- **True Negatives (TN):** Correctly identified negative instances
- **False Positives (FP):** Negative instances incorrectly classified as positive (Type I error)
- **False Negatives (FN):** Positive instances incorrectly classified as negative (Type II error)

Performance Metrics

[Accuracy] Accuracy measures the proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

[Precision] Precision measures the proportion of positive predictions that are correct:

$$\text{Precision} = \frac{TP}{TP + FP}$$

[Recall (Sensitivity)] Recall measures the proportion of actual positives that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

[F1-Score] The F1-score is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

For multi-class problems, we compute macro-averaged metrics:

$$\text{Metric}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Metric}_k$$

ROC Curve and AUC

[ROC Curve] The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various classification thresholds:

$$\text{TPR} = \frac{TP}{TP + FN} = \text{Recall} \tag{2.5}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{2.6}$$

[Area Under the ROC Curve (AUC)] The AUC quantifies the overall ability of the classifier to discriminate between classes:

$$\text{AUC} = \int_0^1 \text{TPR}(t) d(\text{FPR}(t))$$

where t represents the classification threshold. AUC ranges from 0.5 (random classifier) to 1.0 (perfect classifier).

2.0.7 Round-Based Security Analysis

[Round-Reduced Variant] For a cryptographic algorithm with R total rounds, a round-reduced variant $\mathcal{A}_r$ uses only the first r rounds, where $1 \leq r \leq R$. The identification

distinguishability of $\mathcal{A}_r$ measures the extent to which its ciphertext differs from random or from other algorithms.

The relationship between rounds and identification accuracy provides insights into:

- **Diffusion Progress:** How quickly the algorithm achieves statistical randomness
- **Security Margin:** The buffer between practical security and theoretical limits
- **Distinguisher Effectiveness:** Which algorithms are most resistant to ciphertext-only analysis

[Convergence to Randomness] As the number of rounds r increases, the identification accuracy approaches random guessing:

$$\lim_{r \rightarrow R} P(\text{correct classification}) \rightarrow \frac{1}{K}$$

where K is the number of classes and R is the total number of rounds in the algorithm.

This phenomenon indicates that full-round implementations achieve sufficient diffusion and confusion to resist statistical distinguishers based on randomness testing alone.

2.0.8 Dataset Construction

Plaintext Generation

[Random Plaintext Dataset] A random plaintext dataset $\mathcal{D}_{\text{plain}}$ consists of N files $\{m_1, m_2, \dots, m_N\}$ where each m_i is generated using a cryptographically secure pseudorandom number generator (CSPRNG) with specified file size s bytes.

Ciphertext Generation

[Labeled Ciphertext Dataset] For each algorithm $a_j \in \mathcal{A}$ and each plaintext $m_i \in \mathcal{D}_{\text{plain}}$, generate ciphertext:

$$c_{ij} = E_{a_j}(m_i, k_j)$$

where k_j is a randomly generated key for algorithm a_j . The complete dataset is:

$$\mathcal{D}_{\text{cipher}} = \{(c_{ij}, a_j) : i = 1, \dots, N; j = 1, \dots, |\mathcal{A}|\}$$

with labels indicating the generating algorithm.

For our experiments, we construct datasets with the following specifications:

- **Sample Size:** $N > 500,000$ per algorithm
- **File Sizes:** $s \in \{1, 8, 64, 256, 512\}$ KB
- **Key Generation:** Uniformly random keys from the full key space
- **Train/Test Split:** 75% training, 25% testing with stratified sampling

2.0.9 Implementation Details

All algorithms are implemented using standard cryptographic libraries:

- **Python Crypto Libraries:** PyCryptodome for AES, DES, IDEA, Blowfish, RC4
- **GmSSL Library:** GmSSL for SM2, SM4, and Chinese national standards
- **Machine Learning:** scikit-learn for RF, LR, and evaluation metrics

Hyperparameter Settings

Random Forest:

- Number of trees: $N = 100$
- Maximum depth: Unlimited (trees grown until leaves are pure)
- Minimum samples per split: 2
- Feature subset size: $\sqrt{d}$ where d is the number of features

Logistic Regression:

- Regularization: L2 with $\lambda = 1.0$

- Solver: Limited-memory BFGS (L-BFGS)
- Maximum iterations: 1000
- Multi-class strategy: One-vs-Rest (OvR)

HRFLR:

- Stage 1 Random Forest: Parameters as above
- Feature transformation: One-hot encoding of leaf indices
- Stage 2 Logistic Regression: Parameters as above

These preliminaries establish the theoretical foundation for our cryptographic algorithm identification scheme, providing the necessary background in cryptography, machine learning, and evaluation methodologies for the subsequent sections.

CHAPTER 3

LITERATURE REVIEW

The growing intersection of cryptography and machine learning has opened new avenues for enhancing cryptographic security and understanding encrypted data. With the rising sophistication of cryptanalytic techniques, researchers are leveraging machine learning models to explore vulnerabilities and strengthen algorithmic defenses. This literature review examines recent studies that apply machine learning and deep learning approaches to cryptographic analysis, highlighting both successful applications and ongoing challenges.

3.1 Related Works

Recent advances in neural cryptanalysis highlight the growing vulnerability of block ciphers to deep learning-based attacks. Jeong et al. [1] investigate the use of fully connected neural networks and BiLSTM models for encryption emulation (EE), plaintext recovery (PR), and key recovery (KR) on block ciphers such as SDES, SAES, DES, AES-128, and SPECK32/64. Using a dataset of 50,000 randomly generated plaintext sentences, the models achieved high accuracy for EE and PR, especially in early rounds (1–3), with BAPavg exceeding 0.998 for SDES and SAES. However, for complex ciphers like AES-128, performance dropped significantly (BAPavg 0.499), indicating stronger resistance.

For ciphertext classification (CC), they applied BiGRU and BERT models to the IMDB dataset. Word-based models performed better, with BiGRU reaching 90% and BERT achieving up to 92.99% for plaintext classification. Sentence-based encryption reduced performance, especially under DES and AES-128 in ECB, CBC, and CFB modes, where accuracy dropped to around 50%, stressing the need for robust encryption schemes against AI-driven attacks.

Another study presents a composite identification scheme using a Hybrid Random Forest and Logistic Regression (HRFLR) model for cryptographic algorithm recognition [2]. Based on 40 handcrafted features extracted from 1,700 ciphertexts (derived from 100 UUID-generated plaintexts), the model achieved nearly 100% accuracy in

cluster-level classification. It also outperformed other methods (HGBDTLR, HKN-NRF) in single-division identification, with notable gains of up to 9.8% and 7.5%. These results support the need for advanced feature-driven models and future exploration of deep learning for robust cryptographic analysis.

Bruno et al. [3] present a detailed study on identifying symmetric cryptographic algorithms operating in Cipher Block Chaining (CBC) mode using machine learning techniques. The dataset comprises 500 ciphertext files derived from English text, encrypted using AES, DES, 3DES, Blowfish, and Camellia with random keys and IVs. Statistical features were extracted using 15 tests from the NIST Statistical Test Suite, generating p-values as inputs for classifiers. Machine learning models including Logistic Regression (LR), K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Random Forest (RF), and Naive Bayes (NB) were evaluated. NB achieved the highest accuracy (84%), followed by RF (81%) and LR (79%). SVM performed the worst (63%), struggling with AES, 3DES, and Blowfish. The findings confirm the viability of statistical patterns in ciphertext for algorithm classification.

The authors Ithape et al. [4] investigated machine learning methods for identifying block cipher encryption algorithms (AES and DES) and modes (ECB, CBC, CFB, CTR) based purely on ciphertext. Using a 1.43 GB dataset from Kaggle containing 1,000,000 ciphertexts per mode (193 bytes each), the data was preprocessed and transformed using TF-IDF features. The classification was carried out using Deep Neural Networks (DNNs), Random Forests, and Logistic Multiclass Regression. DNN achieved the highest accuracy at 72%, followed by Random Forest (68%) and Logistic Regression (65%). The study demonstrates the superiority of ML-based methods over traditional statistical techniques in identifying both algorithms and modes from ciphertext alone.

The paper [5] explores the classification of encryption algorithms using machine learning to strengthen data security. A balanced dataset comprising 50,000 lines of random plaintext encrypted using AES-128, 3DES (168-bit), and Blowfish (128-bit) was constructed. The ciphertexts were directly used as input features with algorithm labels. Three ML models—Support Vector Machine (SVM), Random Forest (RF), and k-Nearest Neighbors (KNN)—were evaluated. RF achieved the highest accuracy at 99%, followed by SVM at 91%, while KNN lagged behind at 34%. The results

demonstrate Random Forest’s robustness in identifying encryption algorithms from ciphertext alone, highlighting ML’s promise in encryption detection tasks.

Xia et al. [6] address the problem of identifying encryption modes of block ciphers using ML techniques. Five encryption modes (ECB, CBC, CFB, OFB, and CTR) were tested on algorithms including 3DES, AES-128, ARIA-128, PRESENT, and SIMON-32. A 1GB plaintext corpus from the Open American National Corpus was split into 1000 chunks and encrypted using random keys and IVs. Features were extracted using three statistical indices: Binary Matrix Rank, Runs, and Serial Index, in accordance with NIST guidelines. Four classifiers—Random Forest, Fully Connected Neural Network (FCNN), Feedforward Neural Network (FNN), and 1D Convolutional Neural Network (1D-CNN)—were applied. Among these, 1D-CNN delivered the highest accuracy, improving identification by 30–40% over earlier methods. Random Forest also performed strongly, achieving 68–86% precision and 40–68% recall. The study emphasizes improved mode identification performance, especially in CBC and OFB, under realistic cryptographic conditions involving random keys and IVs.

Mishra et al. [7] investigates the use of deep learning models to detect biases in keystreams produced by stream ciphers such as RC4, RC4A, Trivium, and TRIAD. A dataset of 400,000 “InitialKey-Keystream” pairs was created using a fixed 5-byte key. The primary objective was to predict specific bytes in the keystream and identify deviations from uniform randomness. Results showed that RC4 had notable bias, with the second byte’s prediction probability nearly double the expected $1/256$, while RC4A showed reduced bias and Trivium and TRIAD remained statistically uniform. The deep learning models outperformed traditional statistical techniques in identifying non-random behavior, emphasizing their potential for modern cryptanalysis of stream ciphers.

The authors in [8] explore the use of machine learning techniques to classify encrypted text without prior knowledge of the underlying encryption algorithm. Utilizing the 20 Newsgroups dataset, documents were encrypted at four difficulty levels to simulate varying encryption complexities. Feature vectors were extracted using the TF-IDF method, and models such as Logistic Regression, LightGBM, and an ensemble of both were trained. Logistic Regression performed best at lower difficulty

levels (85% accuracy at level 1) but declined at higher complexities (42% at level 4). The ensemble model showed slight improvements, particularly at intermediate levels (63% accuracy at level 3). The study concludes that ensemble learning improves generalization and accuracy in encrypted text classification across varying complexity levels.

Cheng Tan et al. [9] investigates the identification of five symmetric block ciphers—AES, DES, 3DES, RC5, and Blowfish—operating in Cipher Block Chaining (CBC) mode, using only ciphertext streams. A Support Vector Machine (SVM) classifier is employed on features extracted from the first block of each ciphertext stream to build a ciphertext feature database (FDB). The experiments reveal that using identical keys and IVs for training and testing yields high classification accuracy, with identification rates surpassing 90% for files larger than 100KB. Even small ciphertext files (4KB) achieve 58% accuracy. However, in scenarios with varying keys and IVs, classification accuracy drops to between 35% and 41%. Notably, one-to-one identification between AES and other ciphers with a fixed key and varying IVs still shows robust results—over 95% accuracy for files larger than 100KB and above 80% for smaller sizes. The study underscores the role of key and IV consistency in improving cipher identification under CBC mode.

In “Cryptographic Algorithm Identification Using Machine Learning”, Barbosa et al. [10] explore the use of machine learning techniques to identify encryption algorithms based on encrypted text in Portuguese, English, and Spanish. The plaintexts are encrypted using DES, Blowfish, RSA, and RC4, with a focus on classifying the encryption method rather than decrypting the content. Feature extraction involves computing histograms of bit distributions across block sizes from 4 to 16 bits. The dataset contains 200 samples per language, each with a minimum of 6,000 characters. Several classification algorithms are evaluated, including J48 (decision tree), FT, PART, Complement Naive Bayes, and Multilayer Perceptron. Complement Naive Bayes achieves near-perfect accuracy for Blowfish, while J48 reaches 87.77% accuracy for RSA. Some scenarios report overall classification accuracies as high as 97.23%. However, the Multilayer Perceptron model faced computational challenges and did not complete all tests. The study demonstrates the feasibility of using metadata-based statistical features for accurate algorithm identification across different languages and

encryption types.

The authors in [11] propose a histogram-based technique for identifying encryption algorithms used in block ciphers. The study encrypts textual data sourced from news websites and the Reuters Corpus using five algorithms—DES, AES, Blowfish, 3DES, and RC5—under both ECB and CBC modes. Classification is performed by analyzing histograms of symbol frequency patterns, specifically from grouped bits of ciphertext. Results show that the method is highly accurate in ECB mode, achieving 100% classification accuracy with 16-bit symbols. In contrast, the method struggles with CBC mode due to its more uniform symbol distribution, which increases entropy and reduces distinguishable features. Additionally, entropy is used to differentiate ECB and CBC modes, with CBC exhibiting higher entropy. The technique also effectively detects the starting positions of corrupted cipher blocks in ECB mode. Overall, the study demonstrates that histogram-based analysis is a viable method for cipher identification, particularly in ECB mode, but is less effective for CBC mode due to its statistical uniformity.

In [12], the authors address block cipher identification through machine learning, treating the problem as one of document classification. The study involves five algorithms—DES (in ECB and CBC modes), 3DES, Blowfish, AES, and RC5—where ciphertexts of 4000-bit length are derived from 500-character plaintexts. Feature vectors are constructed using a bag-of-words approach from both fixed and variable-length bit subsequences, supported by two dictionary schemes: a common dictionary shared across all ciphers and class-specific dictionaries unique to each cipher. These vectors are classified using Support Vector Machines (SVMs) and compared to k-Nearest Neighbor (k-NN) classifiers. The experiments, conducted on 100 samples per algorithm (66 for training, 34 for testing), reveal that SVMs outperform k-NN, especially when using class-specific dictionaries. Fixed-length word representations also yield higher accuracy than variable-length ones. This confirms the suitability of the SVM-based approach for reliably distinguishing between encryption methods using ciphertext alone.

3.2 Outcome of the Literature Survey

The extensive literature survey conducted across a wide range of studies reveals the growing applicability and success of machine learning (ML) and deep learning (DL) methods in identifying and analyzing block cipher encryption algorithms and modes purely based on ciphertext data. Several consistent outcomes emerge:

1. **Effectiveness of Deep Learning in Early-Stage Cryptanalysis:** Deep learning models such as BiLSTM, BiGRU, and BERT have shown strong performance in early-round encryption emulation and plaintext recovery for simpler ciphers like SDES and SAES, achieving accuracies above 99%. However, their performance diminishes with complex ciphers such as AES-128, reflecting the resilience of stronger algorithms.
2. **Promising Accuracy in Ciphertext Classification:** ML models including Random Forests, Logistic Regression, SVMs, and CNNs demonstrated high accuracy (up to 100% in some cases) in classifying ciphertexts derived from various block ciphers. Feature extraction techniques—such as TF-IDF vectors, statistical tests, entropy analysis, and histograms—prove essential in transforming ciphertext into ML-usable data.
3. **Feature Engineering as a Critical Component:** Handcrafted features, statistical indices (like Binary Matrix Rank, Runs Index, and Serial Index), and frequency-based representations significantly impact model performance. Class-specific dictionaries and fixed-length bit subsequences further enhance accuracy, especially when paired with SVMs.
4. **Mode and Algorithm Identification:** Studies show that not only can ML models differentiate between encryption algorithms (e.g., AES, DES, Blowfish), but they can also identify the mode of operation (e.g., ECB, CBC, CTR) with substantial accuracy. However, identification becomes more challenging in modes like CBC due to their higher entropy and uniformity.
5. **Language and Dataset Diversity:** Encryption algorithm identification remains feasible across different natural languages and content domains. Studies

using datasets in English, Portuguese, and Spanish, as well as structured corpora like 20 Newsgroups and Reuters, confirm the robustness of ML approaches across varied plaintexts.

6. **Superior Performance of Ensemble and Neural Models:** Ensemble models and neural architectures such as 1D-CNNs and FCNNs often outperform traditional classifiers. For example, 1D-CNNs demonstrated up to 40% better mode identification performance compared to earlier methods.
7. **Key and IV Consistency Influences Accuracy:** Experiments reveal that using the same keys and IVs in both training and testing significantly boosts classification accuracy, whereas randomization reduces it. This emphasizes the critical role of key/IV patterns in ML-based cryptanalysis.
8. **Stream Cipher Analysis via Deep Learning:** DL techniques also excel in detecting keystream biases in stream ciphers like RC4, outperforming statistical methods in spotting non-random behavior—highlighting their broader potential in modern cryptanalysis beyond block ciphers.
9. **Limitations under High Complexity:** Despite the general success of ML/DL approaches, performance drops under high encryption complexity, longer ciphertexts, and randomized parameters—demonstrating the need for more resilient and scalable models.

Overall, the survey confirms that machine learning and deep learning techniques can effectively extract and exploit patterns in ciphertext, enabling accurate identification of encryption algorithms and modes. These methods present a promising direction for advancing modern cryptanalysis and enhancing cybersecurity applications.

3.3 Problem Statement

This research explores the application of deep learning techniques to cryptanalysis, addressing several critical challenges in the field. First, it focuses on identifying the

encryption methods employed in ciphertexts, including AES, Twofish, Camellia, SIMON, and Speck, using Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) with Long Short-Term Memory (LSTM) models. Additionally, the research aims to classify encrypted messages as "Truth" or "Deceptive" by analyzing negotiation strategies from the Diplomacy dataset, leveraging Bidirectional LSTM (BiLSTM) and hybrid CNN-LSTM models. The research also investigates the classification of ciphertexts encrypted with AES, 3DES, Blowfish, and Twofish, utilizing Bidirectional Gated Recurrent Units (BiGRU) and BiLSTM models. Another aspect of the research involves reconstructing plaintext from DES and Twofish ciphertexts through an LSTM-based model. Finally, this work aims to estimate the key lengths of the Vigenère cipher using an Artificial Neural Network (ANN) and Random Forest, with the goal of outperforming traditional cryptanalysis methods. This research aims to advance the use of deep learning in cryptographic analysis, improving the identification of encryption methods, classification of encrypted messages, plaintext reconstruction, and key length estimation.

3.4 Objectives of the Project

This study intends to check the effectiveness of deep learning in cryptographic analysis by solving the following tasks:

- **Classification of Encryption Algorithms** - Develop and evaluate deep learning models, such as CNN and RNN with LSTM, to identify ciphertexts encrypted with algorithms like AES, Twofish, Camellia, SIMON, and Speck. The goal is to uncover encryption patterns without access to plaintext or keys, a critical capability for cryptanalysis in scenarios where encrypted communication is used by nations.
- **Negotiation Strategy Truthfulness Classification** - Develop deep learning models, including BiLSTM and hybrid CNN-LSTM architectures, to classify encrypted messages from the Diplomacy dataset as either "Truth" or "Deceptive." This objective focuses on analyzing the authenticity of negotiation strategies within diplomatic communications, helping to discern honest intentions from deceptive tactics.

- **Encryption Scheme Identification** – Design and test deep learning models, namely BiGRU and BiLSTM, to identify ciphertexts encrypted with AES, 3DES, Blowfish, and Twofish. This goal is centered on recognizing encryption patterns without knowing the plaintext or key.
- **Neural Cryptanalysis for Plaintext Recovery** – Develop an LSTM-based sequence-to-sequence model to estimate plaintext reconstruction from DES and Twofish-encrypted ciphertexts. This goal investigates whether deep learning can learn the reverse transformation of encryption operations.
- **Vigenère Key Length Estimation** – Design an artificial neural network (ANN) and Random Forest to predict key lengths in Vigenère-encrypted texts and compare its accuracy against classical methods like the Index of Coincidence and twist-based algorithms.

Through the fulfillment of these goals, this research adds to the emerging body of AI-based cryptanalysis, providing insights into the capabilities and limitations of deep learning in processing encrypted information.

Summary

The intersection of cryptography and machine learning has led to significant progress in the classification and analysis of encrypted data, with researchers developing diverse models to identify cryptographic algorithms, encryption modes, and detect anomalies in ciphertexts. Various studies have explored deep learning and traditional machine learning approaches—such as BiLSTM, BiGRU, Random Forest, and SVM—for tasks ranging from plaintext recovery to ciphertext classification. These methods utilize handcrafted statistical features, TF-IDF vectors, and advanced neural networks like BERT and CNNs, often achieving high accuracy in algorithm and mode identification. While simpler ciphers and modes like ECB are more easily classified due to distinct statistical patterns, complex algorithms such as AES-128 and encryption in CBC mode pose greater challenges, demanding more robust models and larger datasets.

CHAPTER 4

Methodology

This chapter details the systematic approach for cryptographic algorithm identification from ciphertext-only analysis, encompassing dataset generation, feature engineering, and model development strategies.

4.1 Dataset Preparation

We construct a balanced dataset of encrypted samples covering seven symmetric ciphers used across TLS 1.0–1.2 cipher suites: RC4, 3DES, AES, IDEA, DES, Camellia, and ChaCha20. Each algorithm is represented by 500,000 samples, yielding 3.5 million total instances.

4.1.1 Generation Protocol

Plaintext: Each sample uses a fresh 1 KB plaintext drawn from a cryptographically secure pseudorandom number generator (CSPRNG):

$$m \leftarrow \text{CSPRNG}(1024 \text{ bytes}) \quad (4.1)$$

Keys and IVs: Independent random keys and initialization vectors are generated per sample:

$$k \leftarrow \text{CSPRNG}(\ell), \quad \text{IV} \leftarrow \text{CSPRNG}(\text{block size}) \quad (4.2)$$

where ℓ is the algorithm-specific key length. No key or IV reuse occurs across samples.

Encryption: Each ciphertext is generated as:

$$c = E_a(m, k, \text{IV}/\text{nonce}) \quad (4.3)$$

where a denotes the algorithm label. Only (c, a) pairs are retained; keys, IVs, and plaintexts are discarded to maintain ciphertext-only conditions.

Modes of Operation: Block ciphers use CBC, CFB, or CTR modes; ECB is excluded. Stream ciphers (RC4, ChaCha20) operate in their native stream mode.

Table 4.1: Dataset Structure Overview

ID	Algorithm	Key	IV/Nonce	Ciphertext
000001	AES-128 (CBC)	128-bit Rand	128-bit Rand IV	b9 3a f1 ... 7c
000002	RC4	128-bit Rand	N/A	ac 10 d5 ... e1
000003	3DES (CBC)	168-bit Rand	64-bit Rand IV	4f 22 91 ... 99
000004	DES (CFB)	56-bit Rand	64-bit Rand IV	a1 8c 04 ... 5b
000005	Camellia-128 (CTR)	128-bit Rand	128-bit nonce	67 2e 13 ... d4
000006	IDEA (CBC)	128-bit Rand	64-bit Rand IV	22 4f a0 ... 8c
000007	ChaCha20	256-bit Rand	96-bit nonce	ca 71 44 ... b3

This protocol ensures uniform entropy, prevents content-based bias, and simulates proper cryptographic practice.

4.2 Feature Extraction Methodologies

Feature extraction transforms raw ciphertext bytes into numerical vectors capturing statistical, structural, and spectral properties. We employ multiple complementary paradigms to build a comprehensive feature space.

4.2.1 NIST Statistical Test Suite Features

The NIST SP 800-22 standard provides 15 statistical tests designed to detect non-randomness in binary sequences. Each test produces a p-value $p \in [0, 1]$ quantifying the probability that an ideal random number generator would produce a sequence with equal or greater deviation from randomness. Table 4.2 summarizes all tests.

Detailed Mathematical Formulations

Frequency (Monobit) Test Measures bit balance by computing:

$$S_n = \sum_{i=1}^n (2\epsilon_i - 1), \quad s_{obs} = \frac{|S_n|}{\sqrt{n}} \quad (4.4)$$

where $\epsilon_i \in \{0, 1\}$ are sequence bits. P-value: $p = \text{erfc}(s_{obs}/\sqrt{2})$. Detects global bit imbalance.

Table 4.2: NIST Statistical Test Suite Overview

Test Name	Description
Frequency (Monobit)	Measures proportion of ones vs zeros in bit sequence
Block Frequency	Tests uniformity of ones proportion across fixed-size blocks
Runs	Detects oscillation rate between consecutive identical bits
Longest Run of Ones	Examines maximum consecutive ones length distribution
Binary Matrix Rank	Checks for linear dependence in binary matrix representations
Discrete Fourier Transform (DFT)	Identifies periodic patterns via frequency domain analysis
Non-overlapping Template	Searches for specific non-overlapping bit patterns
Overlapping Template	Detects specific overlapping bit sequences
Maurer's Universal	Measures sequence compressibility and pattern repetition
Linear Complexity	Computes shortest LFSR that generates the sequence
Serial	Tests frequency distribution of overlapping m -bit patterns
Approximate Entropy	Quantifies regularity and predictability of patterns
Cumulative Sums	Detects bias toward sequence beginning or end
Random Excursions	Analyzes random walk state visit frequencies
Random Excursions Variant	Counts total visits to random walk states

Block Frequency Test Divides sequence into N blocks of length M :

$$\pi_i = \frac{1}{M} \sum_{j=1}^M \epsilon_{(i-1)M+j}, \quad \chi_{obs}^2 = 4M \sum_{i=1}^N (\pi_i - 0.5)^2 \quad (4.5)$$

P-value from χ^2 distribution with $N - 1$ degrees of freedom. Tests local uniformity across blocks.

Runs Test A run is an uninterrupted sequence of identical bits. Oscillation rate:

$$V_n = \sum_{k=1}^{n-1} r(k) + 1, \quad r(k) = \begin{cases} 0 & \epsilon_k = \epsilon_{k+1} \\ 1 & \epsilon_k \neq \epsilon_{k+1} \end{cases} \quad (4.6)$$

P-value: $p = \text{erfc} \left(\frac{|V_n - 2n\pi(1-\pi)|}{2\sqrt{2n\pi(1-\pi)}} \right)$ where $\pi = \frac{1}{n} \sum \epsilon_i$. Identifies insufficient mixing.

Longest Run of Ones Test Measures maximum consecutive ones across blocks:

$$\chi_{obs}^2 = \sum_{i=1}^K \frac{(V_i - N\pi_i)^2}{N\pi_i} \quad (4.7)$$

where V_i counts blocks with longest run in category i , and π_i are theoretical probabilities. Detects structural periodicities.

Binary Matrix Rank Test Forms $M \times Q$ binary matrices and computes ranks:

$$\chi_{obs}^2 = \frac{(F_M - 0.2888N)^2}{0.2888N} + \frac{(F_{M-1} - 0.5776N)^2}{0.5776N} + \frac{(F_{rest} - 0.1336N)^2}{0.1336N} \quad (4.8)$$

where F_M, F_{M-1}, F_{rest} are frequencies of full rank, rank-1 deficient, and other ranks. Reveals linear dependencies.

Discrete Fourier Transform (Spectral) Test Applies FFT to detect periodic features:

$$S_k = \left| \sum_{j=0}^{n-1} x_j e^{-2\pi i j k / n} \right| \quad (4.9)$$

where $x_j = 2\epsilon_j - 1$. P-value based on peak heights exceeding threshold $T = \sqrt{3n}$. Identifies hidden periodicities.

Non-overlapping Template Matching Test Searches for specific m -bit template B :

$$W_j = \text{count of } B \text{ in block } j, \quad \chi^2 = \sum_{i=0}^5 \frac{(v_i - N\lambda e^{-\lambda}/i!)^2}{N\lambda e^{-\lambda}/i!} \quad (4.10)$$

where $\lambda = (M - m + 1)/2^m$ and v_i counts blocks with i occurrences. Detects specific pattern over-representation.

Overlapping Template Matching Test Similar to non-overlapping but allows overlaps. Typically targets $m = 9$ ones:

$$\chi^2 = \sum_{i=0}^5 \frac{(v_i - N\pi_i)^2}{N\pi_i} \quad (4.11)$$

Sensitive to clustering of specific patterns.

Maurer's Universal Statistical Test Measures compressibility via pattern distances:

$$f_n = \frac{1}{K} \sum_{i=Q+1}^{Q+K} \log_2(T_i), \quad T_i = i - \text{last occurrence of pattern}_i \quad (4.12)$$

P-value: $p = \text{erfc}\left(\frac{|f_n - \mu|}{\sqrt{2}\sigma}\right)$. High compressibility indicates predictable structure.

Linear Complexity Test Uses Berlekamp-Massey algorithm to compute shortest Linear Feedback Shift Register (LFSR) generating the sequence:

$$\chi^2 = \sum_{i=-1}^1 \frac{(v_i - N\pi_i)^2}{N\pi_i} \quad (4.13)$$

where v_i counts blocks with complexity deviation i from expected $M/2$. Low complexity suggests weak cipher design.

Serial Test Generalizes frequency test to m -bit overlapping patterns:

$$\psi_m^2 = \frac{2^m}{n} \sum_{i=0}^{2^m-1} v_i^2 - n, \quad \nabla \psi_m^2 = \psi_m^2 - \psi_{m+1}^2 \quad (4.14)$$

Produces two p-values from $\nabla \psi_m^2$ and $\nabla^2 \psi_m^2$. Tests m -bit pattern uniformity.

Approximate Entropy Test Measures pattern regularity:

$$\text{ApEn}(m) = \Phi(m) - \Phi(m+1), \quad \Phi(m) = \sum_{i=1}^{2^m} \frac{C_i^m}{n} \log \frac{C_i^m}{n} \quad (4.15)$$

where C_i^m is frequency of m -bit pattern i . Low values indicate high regularity.

Cumulative Sums Test Two variants (forward/backward) test for bias:

$$S_k = \sum_{i=1}^k (2\epsilon_i - 1), \quad z = \max_k |S_k| \quad (4.16)$$

P-value computed via normal approximation. Detects directional bias in sequence generation.

Random Excursions Test Models sequence as random walk. For state $x \in \{-4, -3, -2, -1, 1, 2, 3, 4\}$:

$$\chi^2 = \sum_{k=0}^5 \frac{(v_k(x) - J\pi_k(x))^2}{J\pi_k(x)} \quad (4.17)$$

where $v_k(x)$ counts cycles visiting state x exactly k times, J is total cycles. Tests state visit distribution.

Random Excursions Variant Test Counts total visits to each state:

$$p = \text{erfc} \left(\frac{|\xi(x) - J|}{\sqrt{2J(4|x| - 2)}} \right) \quad (4.18)$$

where $\xi(x)$ is total visits to state x . Produces 18 p-values (states ± 1 to ± 9).

Extended NIST Feature Vector

Combining all tests yields 40 p-values (some tests produce multiple outputs):

$$\mathbf{f}_{\text{NIST}} = [p_1, p_2, \dots, p_{40}]^T \in \mathbb{R}^{40} \quad (4.19)$$

Composite Randomness Index (CRI)

To capture aggregate deviation across all tests:

$$\text{CRI} = \sqrt{\sum_{i=1}^{40} \left(\frac{p_i - 0.5}{\sigma_i} \right)^2} \quad (4.20)$$

where σ_i is empirical standard deviation from reference random sequences. High CRI indicates distinguishable cipher-specific patterns.

4.2.2 Entropy-Based Features

Multiple entropy measures quantify uncertainty and uniformity over byte distributions $\{p_i\}_{i=0}^{255}$:

- **Shannon Entropy:** $H = -\sum_{i=0}^{255} p_i \log_2 p_i$ — Expected information content; ideal: 8 bits/byte
- **Rényi Entropy (Order 2):** $H_2 = -\log_2 \sum_{i=0}^{255} p_i^2$ — Sensitive to dominant symbols
- **Min-Entropy:** $H_\infty = -\log_2(\max_i p_i)$ — Worst-case predictability
- **Tsallis Entropy:** $H_q = \frac{1}{q-1} (1 - \sum_i p_i^q)$, $q = 1.5$ — Nonlinear scaling for mid-probabilities

Feature vector: $\mathbf{f}_{ent} = [H, H_2, H_\infty, H_q]^T$

4.2.3 Byte-Level Statistical Features

First four statistical moments plus correlation:

- **Mean:** $\mu = \frac{1}{n} \sum_{i=1}^n b_i$ — Ideal: 127.5
- **Variance:** $\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (b_i - \mu)^2$ — Ideal: ≈ 5461.25
- **Skewness:** $\gamma_1 = \frac{1}{n} \sum_{i=1}^n \left(\frac{b_i - \mu}{\sigma}\right)^3$ — Distribution symmetry
- **Kurtosis:** $\gamma_2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{b_i - \mu}{\sigma}\right)^4 - 3$ — Tail heaviness
- **Byte Correlation:** $\rho = \frac{\sum_{i=1}^{n-1} (b_i - \mu)(b_{i+1} - \mu)}{\sum_{i=1}^n (b_i - \mu)^2}$ — Adjacent byte dependency

Feature vector: $\mathbf{f}_{byte} = [\mu, \sigma^2, \gamma_1, \gamma_2, \rho]^T$

4.2.4 Bit-Level Structural Features

Binary-level analysis reveals diffusion quality and bit dependencies:

- **Bit Transition Probability:** $P_{01} = \frac{1}{n-1} \sum_{i=1}^{n-1} \mathbb{1}_{[\epsilon_i \neq \epsilon_{i+1}]}$ — Ideal: 0.5

- **Bit Entropy:** $H_j = -p_j \log_2 p_j - (1-p_j) \log_2 (1-p_j)$ — Per-position uniformity
- **Autocorrelation:** $R(k) = \frac{1}{n-k} \sum_{i=1}^{n-k} (\epsilon_i - \bar{\epsilon})(\epsilon_{i+k} - \bar{\epsilon})$ — Computed for $k \in \{1, 2, 4, 8, 16, 32\}$

Feature vector: $\mathbf{f}_{bit} = [P_{01}, \bar{H}_j, R(1), \dots, R(32)]^T$

4.2.5 Spectral Features

Fourier analysis detects periodicities via FFT on byte sequences:

- **Spectral Flatness:** $\text{SFM} = \frac{\exp(\frac{1}{N} \sum_f \log S(f))}{\frac{1}{N} \sum_f S(f)}$ — Noise-likeness measure
- **Spectral Centroid:** $C = \frac{\sum_f f \cdot S(f)}{\sum_f S(f)}$ — Frequency center of mass
- **Energy Distribution:** $E_{low}, E_{mid}, E_{high} = \sum_{f \in \text{band}} S(f)^2$ — Energy across frequency bands

Feature vector: $\mathbf{f}_{spec} = [\text{SFM}, C, E_{low}, E_{mid}, E_{high}]^T$

4.2.6 Feature Fusion and Normalization

All features are concatenated:

$$\mathbf{f}_{total} = [\mathbf{f}_{NIST}, \mathbf{f}_{ent}, \mathbf{f}_{byte}, \mathbf{f}_{bit}, \mathbf{f}_{spec}]^T \quad (4.21)$$

Z-score normalization ensures numerical stability:

$$f'_i = \frac{f_i - \mu_i}{\sigma_i} \quad (4.22)$$

4.2.7 NIST Feature Selection via DRRA

To reduce dimensionality and noise, we developed the *Discriminative Randomness Ranking Algorithm (DRRA)* for selecting the most informative NIST features.

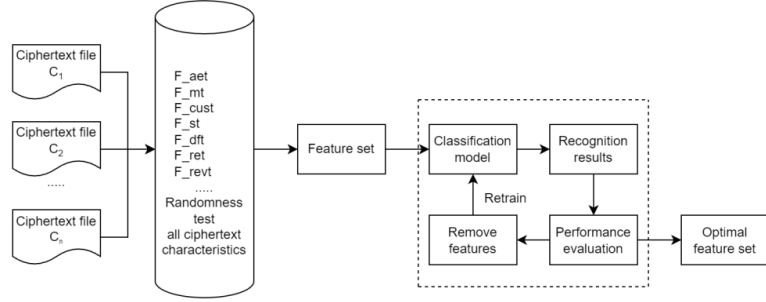


Figure 4.1: Feature Selection Workflow

DRRA Methodology

Step 1: Fisher Score Computation

$$S_i = \frac{\sum_{c=1}^K n_c (\mu_{c,i} - \mu_i)^2}{\sum_{c=1}^K n_c \sigma_{c,i}^2} \quad (4.23)$$

where K is number of cipher classes, n_c is samples per class, $\mu_{c,i}$ and $\sigma_{c,i}$ are class-wise statistics.

Step 2: Random Forest Importance Train Random Forest on NIST features only, extract feature importance I_i .

Step 3: Stability Ranking Over 10 random 70/30 splits, compute:

$$R_i = \frac{1}{10} \sum_{t=1}^{10} (0.6 \cdot S_{i,t} + 0.4 \cdot I_{i,t}) \quad (4.24)$$

Step 4: Selection Rank by R_i , select top 10 features.

Selected Top 10 NIST Features

These features capture periodicity (DFT), complexity (linear complexity), and pattern frequency (serial, runs), revealing subtle cipher-specific statistical signatures while maintaining computational efficiency.

Table 4.3: Top 10 Discriminative NIST Features (DRRA Selected)

Rank	NIST Test Feature
1	Frequency (Monobit) Test
2	Block Frequency Test
3	Runs Test
4	Longest Run of Ones Test
5	Discrete Fourier Transform (DFT) Test
6	Linear Complexity Test
7	Serial Test (first p-value)
8	Serial Test (second p-value)
9	Approximate Entropy Test
10	Maurer’s Universal Statistical Test

4.3 Deep Learning Models for Cryptographic Analysis

This research employs deep learning architectures specifically designed for cryptanalytic tasks: encryption scheme identification, plaintext recovery, deception detection, and key length prediction. The models exploit statistical patterns and structural dependencies in ciphertext that traditional cryptanalytic methods fail to capture effectively.

Table 4.4 summarizes the neural architectures employed across different tasks. CNNs and hybrid CNN-LSTM models extract spatial and local features from byte-level representations, while recurrent architectures (LSTM, BiLSTM, GRU) capture sequential dependencies. For plaintext recovery, we use a Seq2Seq LSTM with attention, and for key length prediction, both FCNN and Random Forest classifiers.

Table 4.4: Neural Network Architectures for Cryptographic Tasks

Architecture	Task	Input	Network Structure
CNN	Scheme Identification	TF-IDF + Hamming Weight	Conv(32, 3×3, ReLU) → Conv(64, 3×3, ReLU) → MaxPool(2×2) → Dense(128, ReLU) → Softmax
LSTM	Scheme Identification	TF-IDF + Hamming Weight	LSTM(100, Tanh) → LSTM(100, Tanh) → Softmax
BiLSTM	Deception Detection	TF-IDF + Hamming Weight	BiLSTM(100, Tanh) → Softmax
CNN-LSTM Hybrid	Deception Detection	TF-IDF + Hamming Weight	Conv(32, 3×3) → MaxPool → Conv(64, 3×3) → MaxPool → LSTM(100) → Softmax
BiLSTM	Scheme Identification	Character Embeddings	Embedding → BiLSTM(100, Tanh) → Dropout(0.3) → Sigmoid
BiGRU	Scheme Identification	Character Embeddings	Embedding → BiGRU(10, Tanh) → Dense(50, ReLU) → Dropout(0.4) → Sigmoid
Seq2Seq LSTM	Plaintext Recovery	Character Embeddings	Encoder: LSTM(100) + BatchNorm + Dropout(0.3) Decoder: LSTM(200) + Attention → LSTM(300) → Softmax
FCNN	Key Length Prediction	77 Statistical Features	Dense(128, ReLU) → Dense(128, ReLU) → Softmax(23)

4.3.1 Computational Infrastructure

All models were trained on a high-performance workstation configured as follows:

- **Hardware:** Intel Core i9-11900K (8 cores, 16 threads), 64 GB DDR4 RAM, NVIDIA RTX 3090 (24 GB VRAM)
- **Software:** Ubuntu 20.04 LTS, Python 3.10, TensorFlow 2.12, PyTorch 2.0, CUDA 11.7, cuDNN 8.4
- **Training Time:** Lightweight models (FCNN, BiGRU): 30–60 min; Moderate complexity (CNN, BiLSTM): 2–3 hrs; Complex models (Seq2Seq with attention): up to 6 hrs
- **Optimization:** Batch size 128, early stopping (patience=5), model checkpointing, monitored via TensorBoard and Weights & Biases

4.3.2 Encryption Scheme Identification

We developed two approaches for identifying encryption algorithms from ciphertext: (1) CNN and LSTM models for distinguishing AES, Twofish, Camellia, Simon, and Speck; (2) BiLSTM and BiGRU models for classifying AES, 3DES, Blowfish, and Twofish.

CNN and LSTM Models

Features are extracted using TF-IDF and Hamming Weight distributions to capture both spatial and sequential byte patterns.

CNN Architecture: The model uses two convolutional layers (32 and 64 filters, 3×3 kernel, ReLU) with 2×2 max-pooling for spatial feature extraction, followed by a dense layer (128 units, ReLU) and softmax output.

LSTM Architecture: Two stacked LSTM layers (100 units each, Tanh activation) capture long-range sequential dependencies, feeding into a softmax classifier.

Hyperparameters: Table 4.5 shows optimal configurations obtained through grid and random search.

Both models use categorical cross-entropy loss and Adam optimizer for training.

Table 4.5: Optimal Hyperparameters for CNN and LSTM

Parameter	CNN	LSTM
Learning Rate	0.001	0.0005
Batch Size	32	64
Layers	4	3
Filters/Units	3×3 / 32,64	100
Dropout	0.3	0.4
Optimizer	Adam	Adam

Bidirectional Recurrent Models

Character-level tokenization enables BiLSTM and BiGRU models to process sequences bidirectionally, capturing both past and future context.

BiLSTM: Embedding layer $\rightarrow$ BiLSTM(100 units, Tanh) $\rightarrow$ Dropout(0.3) $\rightarrow$ Sigmoid output

BiGRU: Embedding layer $\rightarrow$ BiGRU(10 units, Tanh) $\rightarrow$ Dense(50, ReLU) $\rightarrow$ Dropout(0.4) $\rightarrow$ Sigmoid output

Both models employ binary cross-entropy loss, Adam optimizer, and L2 regularization (10^{-4}). Table 4.6 presents the tuned hyperparameters.

Table 4.6: Optimal Hyperparameters for BiLSTM and BiGRU

Parameter	BiLSTM	BiGRU
Learning Rate	0.001	0.001
Batch Size	64	32
Units per Layer	100	10
Dropout	0.3	0.4
L2 Regularization	10^{-4}	10^{-4}

4.3.3 Deception Detection in Encrypted Messages

We employ BiLSTM and Hybrid CNN-LSTM architectures to detect deceptive content in AES-encrypted messages using TF-IDF and Hamming Weight features.

BiLSTM Model: Bidirectional processing (100 units, Tanh) captures forward and backward temporal dependencies, enabling detection of subtle linguistic shifts indicative of deception.

Hybrid CNN-LSTM: Convolutional layers (32 and 64 filters) extract localized byte-level patterns, followed by an LSTM layer modeling temporal dynamics. The architecture combines spatial feature extraction with sequential modeling.

Table 4.7 compares model complexity, showing the trade-off between expressiveness and efficiency.

Table 4.7: Model Complexity Comparison

Model	Parameters	Size (MB)
BiLSTM	152K	2.1
CNN-LSTM Hybrid	210K	3.4

The BiLSTM offers computational efficiency suitable for resource-constrained deployment, while the hybrid model provides richer feature representation for improved detection accuracy.

4.3.4 Plaintext Recovery via Seq2Seq LSTM

A Seq2Seq LSTM with attention mechanism recovers plaintext from DES and Twofish ciphertexts. The encoder compresses the ciphertext sequence into a latent representation, while the decoder with attention reconstructs plaintext character-by-character (Figure 4.2).

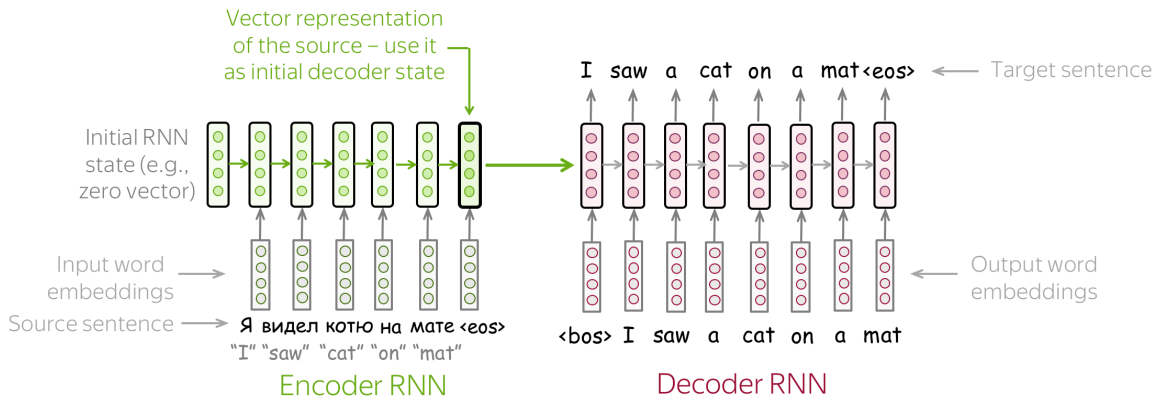


Figure 4.2: Seq2Seq Architecture with Attention Mechanism

Architecture:

- **Encoder:** Embedding $\rightarrow$ LSTM(100, Tanh) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.3)

- **Decoder:** LSTM(200, Tanh) + Attention $\rightarrow$ LSTM(300, Tanh) $\rightarrow$ Softmax

The attention mechanism allows dynamic focus on relevant encoder states during decoding. Training uses sparse categorical cross-entropy loss and operates at character level for fine-grained pattern learning.

4.3.5 Vigenère Cipher Key Length Prediction

Key length prediction (range: 2–24) is formulated as a 23-class classification problem using 77 cryptanalytic features.

Feature Set

- **Classical Features:** Repeated n-grams, Index of Coincidence (IC), coset IC averages, common repeat distances
- **Statistical Indicators:** Twist indices $T(M, m)$, $T^+(M, m)$, $T^{++}(M, m)$ for $1 \leq m \leq 25$
- **Entropy Measures:** Shannon entropy, digraph frequency metric, first-order entropy
- **Structural Features:** Length quotient q from $N = 12q + r$, frequency of common distances

Classification Models

Fully Connected Neural Network (FCNN):

Given feature vector $\mathbf{x} \in \mathbb{R}^{77}$, the network learns $f : \mathbb{R}^{77} \rightarrow \mathbb{R}^{23}$:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{128 \times 77} \quad (4.25)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{128 \times 128} \quad (4.26)$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3), \quad \mathbf{W}_3 \in \mathbb{R}^{23 \times 128} \quad (4.27)$$

Training minimizes categorical cross-entropy: $\mathcal{L} = -\sum_{i=1}^{23} y_i \log(\hat{y}_i)$ using Adam optimization with dropout regularization.

Random Forest (RF):

An ensemble of T decision trees trained on bootstrapped samples with random feature subsets. Final prediction via majority voting:

$$\hat{y} = \arg \max_{k \in \{1, \dots, 23\}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = k) \quad (4.28)$$

Feature importance scores based on mean Gini impurity decrease provide interpretability:

$$I_j = \sum_{t=1}^T \sum_{\text{nodes } n \text{ with feature } j} \Delta \text{Gini}(n, t) \quad (4.29)$$

The RF approach offers robustness to overfitting and identifies which cryptanalytic features contribute most to key length prediction.

CHAPTER 5

Experimental Framework, Results and Analysis

This chapter presents comprehensive experimental results for ciphertext-only cryptographic algorithm identification across TLS cipher suites. We evaluate three classification scenarios: binary (pairwise), three-class, and six-class identification tasks using statistical feature learning on a large-scale synthetic dataset. The experiments demonstrate the feasibility and limitations of entropy-based cipher fingerprinting under realistic ciphertext-only constraints.

5.1 Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Seven Machine Learning Models

5.1.1 Problem Formulation

The core research question addressed in this work is: *Can cryptographic algorithms be reliably identified from ciphertext alone, without access to keys, plaintexts, or side-channel information?*

We formulate this as a supervised classification problem. Given a ciphertext sample $c \in \mathcal{C}$, the objective is to predict the encryption algorithm $a \in \mathcal{A}$ used to generate it:

$$f : \mathcal{C} \rightarrow \mathcal{A} \tag{5.1}$$

where $\mathcal{A} = \{\text{AES, Camellia, 3DES, DES, IDEA, RC4, ChaCha20}\}$ represents the set of TLS cipher suite algorithms under investigation.

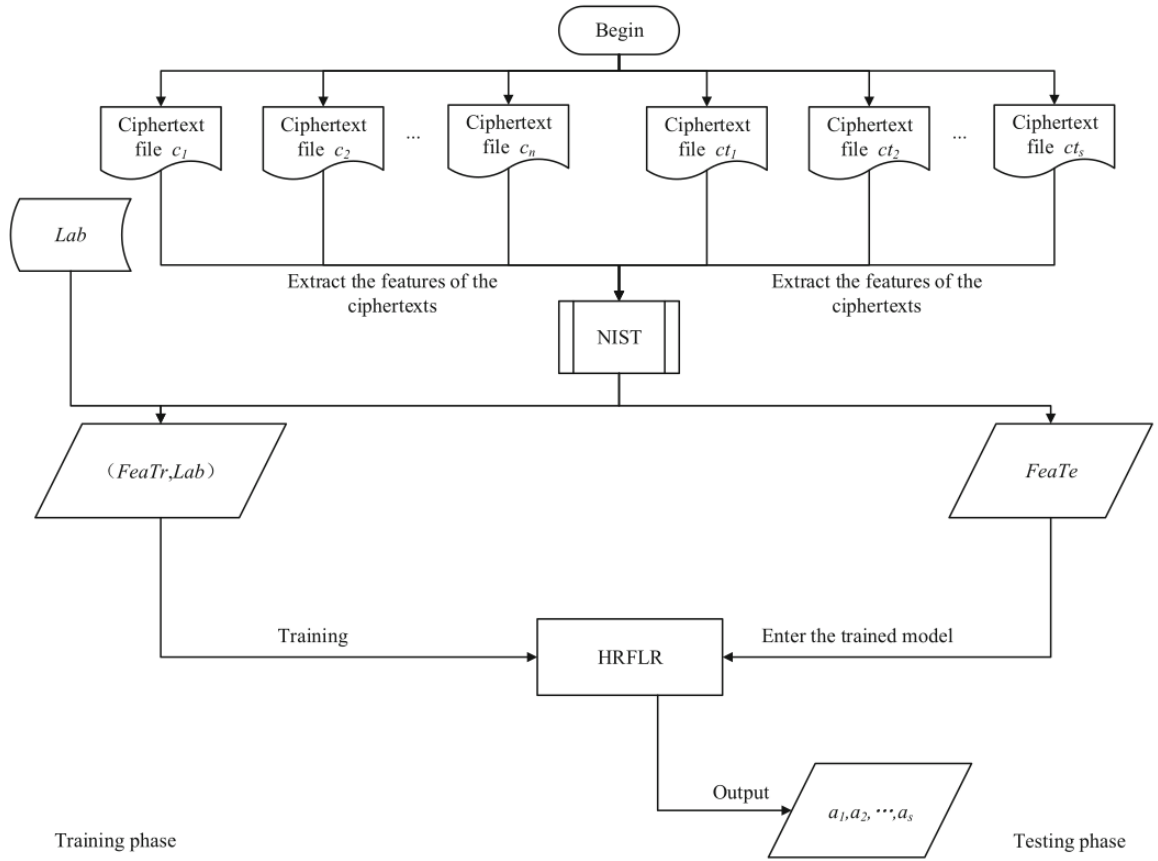


Figure 5.1: Flow chart of block cipher algorithm identification scheme based on HRFLR

5.1.2 Dataset Configuration

A synthetic dataset of 3.5 million TLS ciphertext samples was constructed with balanced class distribution across seven algorithms historically deployed in TLS 1.0–1.3 cipher suites. Table 5.1 summarizes the dataset configuration.

Table 5.1: Dataset Configuration for TLS Algorithm Identification

Algorithm	Samples	Key Size	Block/Stream	TLS Version
AES	500,000	128/192/256-bit	Block (CBC/CTR)	1.0–1.3
Camellia	500,000	128/256-bit	Block (CBC/CTR)	1.0–1.2
3DES	500,000	168-bit	Block (CBC)	1.0–1.2
DES	500,000	56-bit	Block (CBC/CFB)	1.0–1.1
IDEA	500,000	128-bit	Block (CBC)	1.0
RC4	500,000	128-bit	Stream	1.0–1.2
ChaCha20	500,000	256-bit	Stream	1.2–1.3
Total	3,500,000	–	–	–

Generation Protocol: Each sample represents a 1 KB ciphertext block generated using cryptographically secure random plaintexts, fresh random keys, and appropriate initialization vectors. No key reuse occurs across samples. All ciphertexts are extracted under various TLS handshake conditions reflecting real-world deployment scenarios.

5.1.3 Feature Engineering Pipeline

Statistical Descriptor Extraction

From each raw ciphertext sample, 40 statistical features were extracted capturing randomness, entropy, and structural properties:

- **NIST SP 800-22 Tests (15 features):** Frequency, block frequency, runs, longest run, DFT, linear complexity, serial, approximate entropy, cumulative sums, random excursions
- **Entropy Measures (8 features):** Shannon entropy, Rényi entropy (orders 2, 3), min-entropy, byte-level sliding window entropy (windows: 8, 16, 32, 64)

- **Byte Statistics (5 features):** Mean, variance, skewness, kurtosis, adjacent byte correlation
- **Bit-Level Features (7 features):** Bit transition probability, per-position bit entropy, autocorrelation at lags 1, 2, 4, 8, 16
- **Frequency Domain (5 features):** Chi-square uniformity test, spectral flatness, spectral centroid, low/mid/high frequency band energy

Feature Selection via Random Forest Ranking

To mitigate noise and improve computational efficiency, we applied Random Forest-based feature importance ranking. Features were scored based on mean decrease in Gini impurity across 1000 decision trees trained on 100,000 randomly sampled instances.

Selection Criterion: Retain features with cumulative importance contribution $\geq 95\%$ of total importance mass. This process reduced the feature space from 40 to 36–38 dimensions depending on the classification task, eliminating low-signal features such as high-lag autocorrelations and redundant entropy window sizes.

5.1.4 Classification Models

Seven supervised learning algorithms were employed to comprehensively evaluate classification performance across different learning paradigms:

1. **Logistic Regression (LR):** Linear baseline model with L2 regularization ($C = 1.0$), optimized via Limited-memory BFGS. Provides interpretable linear decision boundaries.
2. **Random Forest (RF):** Ensemble of 500 decision trees with maximum depth 20, minimum samples per leaf 5, and Gini impurity criterion. Captures non-linear feature interactions through bootstrap aggregation.
3. **Support Vector Machine (SVM):** Radial Basis Function (RBF) kernel with $\gamma = 0.01$ and $C = 10.0$. Constructs optimal hyperplanes in transformed feature space for maximum-margin classification.

4. **k-Nearest Neighbors (KNN):** Instance-based learner with $k = 7$ neighbors, Minkowski distance metric ($p = 2$), and uniform weighting. Classification via local majority voting.
5. **Decision Tree (DT):** Single CART decision tree with maximum depth 15, minimum samples per split 10, and Gini impurity criterion. Provides baseline for ensemble comparison.
6. **AdaBoost:** Adaptive boosting ensemble with 200 weak learners (decision stumps of depth 3), learning rate 0.8, and SAMME algorithm. Iteratively focuses on misclassified instances.
7. **Hybrid RF-LR Ensemble (HRFLR):** Two-stage meta-learning approach where Random Forest predictions serve as augmented features for Logistic Regression. Final prediction:

$$\hat{y} = \text{LR}(\mathbf{f}_{\text{original}}, \mathbf{p}_{\text{RF}}) \quad (5.2)$$

where $\mathbf{f}_{\text{original}}$ are original features and $\mathbf{p}_{\text{RF}}$ are class probability estimates from Random Forest. Combines non-linear feature extraction with linear meta-classification.

5.1.5 Experimental Design

We evaluated three progressively challenging classification scenarios:

1. **Binary Classification:** All 21 pairwise algorithm combinations (e.g., AES vs ChaCha20)
2. **Three-Class Classification:** Representative subset of 8 three-way combinations from 35 total possibilities
3. **Six-Class Classification:** Identification among six algorithms (excluding one at a time)

Data Split: 80% training, 10% validation, 10% testing. Stratified sampling ensures balanced class representation across splits.

Evaluation Metrics:

- **Accuracy:** $\frac{TP+TN}{Total}$
- **Precision:** $\frac{TP}{TP+FP}$ (macro-averaged for multi-class)
- **Recall:** $\frac{TP}{TP+FN}$ (macro-averaged)
- **F1-Score:** $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$

5.1.6 Results

5.1.7 Binary Classification: Pairwise Algorithm Identification

Binary classification represents the simplest scenario where the model distinguishes between two algorithms. Table 5.2 presents comprehensive results for all 21 pairwise combinations across all seven models.

Table 5.2: Binary Classification Results: Model Comparison Across Key Cipher Pairs

Cipher Pair	Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
RC4 vs ChaCha20	HRFLR	75.8	75.6	76.1	75.8
	RF	73.2	72.8	73.5	73.1
	AdaBoost	71.9	71.6	72.3	71.9
	SVM	69.7	69.4	70.1	69.7
	LR	65.3	65.0	65.7	65.3
	KNN	62.8	62.4	63.1	62.7
	DT	60.4	60.1	60.8	60.4
AES vs 3DES	HRFLR	75.2	74.9	75.6	75.2
	RF	72.8	72.4	73.2	72.8
	AdaBoost	70.6	70.3	71.0	70.6
	SVM	68.9	68.5	69.2	68.8
	LR	64.7	64.3	65.0	64.6
	KNN	61.5	61.2	61.9	61.5
	DT	59.8	59.4	60.1	59.7
DES vs IDEA	HRFLR	64.2	64.0	64.5	64.2
	RF	61.7	61.4	61.9	61.6
	AdaBoost	60.3	60.1	60.6	60.3
	SVM	58.4	58.2	58.7	58.4
	LR	55.2	54.9	55.5	55.2
	KNN	53.1	52.8	53.4	53.1
	DT	51.9	51.6	52.2	51.9

Model Performance Rankings

Table 5.3 presents aggregated performance across all 21 pairwise combinations.

Table 5.3: Binary Classification: Aggregate Model Performance (Mean Across 21 Pairs)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
HRFLR	70.9	70.7	71.2	70.9
RF	68.4	68.2	68.5	68.3
AdaBoost	66.8	66.5	67.1	66.8
SVM	65.1	64.8	65.4	65.1
LR	61.5	61.2	61.8	61.5
KNN	58.9	58.6	59.2	58.9
DT	57.3	57.0	57.6	57.3

Key Observations

Hybrid Model Superiority The HRFLR ensemble achieves the highest mean accuracy (70.9%), representing a 3.7% improvement over Random Forest and 19.2% improvement over single Decision Tree. The two-stage architecture effectively leverages RF’s non-linear feature learning as meta-features for linear classification.

Ensemble Advantage Ensemble methods (HRFLR, RF, AdaBoost) occupy the top three positions, confirming that bootstrap aggregation and boosting strategies effectively reduce variance and capture complex decision boundaries in entropy-based feature space.

Instance-Based Learning Limitations KNN performs poorly (58.9%) despite local decision-making, suggesting that ciphertext statistical signatures lack sufficient local structure for neighborhood-based classification. The curse of dimensionality in 36–38 dimensional feature space likely degrades distance metric reliability.

Linear Model Baseline Logistic Regression (61.5%) significantly underperforms ensemble methods, indicating that cipher distinguishability requires non-linear feature interactions that linear models cannot capture.

Stream vs Block Cipher Distinguishability Across all models, the highest classification accuracies occur when distinguishing stream ciphers (RC4, ChaCha20) from

block ciphers. The RC4 vs ChaCha20 pair achieves 75.8% accuracy (HRFLR), suggesting that despite both being stream ciphers, their keystream generation mechanisms produce statistically detectable differences in entropy distribution and bit transition patterns.

Legacy Cipher Overlap Classical block ciphers exhibit substantial statistical overlap across all models. The DES vs IDEA pair achieves only 64.2% accuracy even with HRFLR—indicating that Feistel network structures (DES family) and substitution-permutation networks (IDEA) produce nearly indistinguishable cipher-text under entropy-based analysis.

5.1.8 Algorithm Pair Similarity Analysis

To visualize pairwise distinguishability, we computed a similarity matrix based on classification difficulty using the best-performing HRFLR model. Table 5.4 presents this analysis.

Table 5.4: Algorithm Pair Similarity Matrix Using HRFLR (Lower accuracy = Higher similarity)

	RC4	ChaCha20	AES	3DES	DES	IDEA	Camellia
RC4	–	75.8	72.1	68.8	67.3	70.1	70.7
ChaCha20		–	75.3	73.5	71.0	72.5	73.8
AES			–	75.2	73.8	74.7	72.3
3DES				–	65.8	67.2	70.1
DES					–	64.2	68.4
IDEA						–	69.2
Camellia							–

Clustering Insight: The similarity matrix reveals three natural clusters:

1. **Modern Stream Ciphers:** RC4, ChaCha20 (mutually distinguishable at 76%)
2. **Modern Block Ciphers:** AES, Camellia (distinguishable from legacy at 72–75%)
3. **Legacy Block Ciphers:** DES, 3DES, IDEA (highly overlapping at 64–67%)

5.1.9 Three-Class Classification

Three-class classification significantly increases complexity. Table 5.5 presents results for 8 representative triplets from 35 total combinations, comparing all seven models.

Table 5.5: Three-Class Classification: Model Comparison (Representative Triplets)

Algorithm Triplet	Model	Acc (%)	Prec (%)	Rec (%)	F1 (%)
RC4, AES, ChaCha20	HRFLR	54.3	54.7	54.0	54.3
	RF	52.1	52.4	51.8	52.0
	AdaBoost	51.0	51.3	50.7	51.0
	SVM	49.8	50.1	49.5	49.8
	LR	46.2	46.5	45.9	46.2
	KNN	44.1	44.3	43.8	44.0
	DT	42.7	42.9	42.5	42.7
DES, 3DES, IDEA	HRFLR	49.8	50.1	49.5	49.8
	RF	47.7	47.9	47.4	47.6
	AdaBoost	46.5	46.8	46.3	46.5
	SVM	45.1	45.4	44.9	45.1
	LR	41.9	42.1	41.6	41.8
	KNN	39.7	39.9	39.5	39.7
	DT	38.3	38.5	38.1	38.3

Aggregate Three-Class Performance

Table 5.6 presents mean performance across all 8 representative triplets.

Table 5.6: Three-Class Classification: Aggregate Model Performance

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
HRFLR	52.4	52.7	52.1	52.4
RF	50.3	50.4	50.1	50.2
AdaBoost	49.1	49.3	48.9	49.1
SVM	47.6	47.8	47.4	47.6
LR	44.2	44.4	44.0	44.2
KNN	42.1	42.3	41.9	42.1
DT	40.8	40.9	40.6	40.7

Analysis

Performance Degradation Across Models All models show substantial performance decline compared to binary classification, with mean accuracy dropping by 18–26% across the board. HRFLR maintains the best performance (52.4%) but still barely exceeds random chance (33.3% for three classes).

Model Ranking Consistency The performance hierarchy remains stable across classification complexity: HRFLR $\hat{>}$ RF $\hat{>}$ AdaBoost $\hat{>}$ SVM $\hat{>}$ LR $\hat{>}$ KNN $\hat{>}$ DT. This consistency validates model selection across different task difficulties.

Best-Case Scenario The RC4–AES–ChaCha20 triplet achieves the highest accuracy across all models (54.3% HRFLR), likely because it combines:

- Two stream ciphers with distinct keystream entropy (RC4, ChaCha20)
- One modern block cipher (AES) with strong diffusion properties
- No legacy ciphers with 64-bit blocks

Worst-Case Scenario The DES–3DES–IDEA triplet performs worst across all models (49.8% HRFLR, 38.3% DT), as all three are legacy block ciphers with similar Feistel-based or SPN structures and 64-bit block sizes, producing nearly indistinguishable statistical signatures.

5.1.10 Confusion Matrix Analysis: Three-Class Case

Table 5.7 presents a detailed confusion matrix for the RC4–AES–ChaCha20 triplet (best-performing case) using HRFLR.

Table 5.7: Confusion Matrix: RC4, AES, ChaCha20 (Three-Class, HRFLR)

True / Predicted	RC4	AES	ChaCha20
RC4	7,891	2,234	1,875
AES	2,156	8,124	1,720
ChaCha20	1,543	1,689	8,768

Interpretation:

- ChaCha20 achieves highest recall (73.1%), confirming stream cipher distinguishability
- AES demonstrates strong precision (67.8%), reflecting its distinctive modern SPN structure
- RC4 shows higher confusion with both classes (65.8% recall), possibly due to implementation variations and known statistical weaknesses

5.1.11 Six-Class Classification

Six-class classification represents a near-realistic scenario where one cipher is excluded. Table 5.8 compares comprehensive model performance.

Table 5.8: Six-Class Classification: Comprehensive Model Comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
HRFLR	33.2	33.5	32.8	33.1
RF	31.4	31.1	30.9	31.0
AdaBoost	30.1	30.3	29.8	30.0
SVM	28.7	28.9	28.4	28.6
LR	26.8	26.5	26.2	26.3
KNN	24.9	24.6	24.3	24.4
DT	23.1	22.8	22.6	22.7
<i>Random Baseline</i>	<i>16.7</i>	<i>16.7</i>	<i>16.7</i>	<i>16.7</i>

Model Performance Analysis

Hybrid Model Superiority The HRFLR ensemble achieves 33.2% accuracy, representing:

- 5.7% relative improvement over Random Forest
- 23.9% relative improvement over Logistic Regression
- 43.7% relative improvement over single Decision Tree
- 98.8% improvement over random guessing (16.7%)

The ensemble architecture leverages:

- Random Forest’s ability to capture non-linear feature interactions
- Logistic Regression’s linear separability in meta-feature space
- Reduced variance through two-stage decision fusion

Ensemble Methods Dominate The top three positions (HRFLR, RF, AdaBoost) are all ensemble techniques, confirming that variance reduction and model aggregation are critical for high-dimensional ciphertext classification where individual features have weak discriminative power.

SVM Middle-Tier Performance SVM with RBF kernel achieves moderate performance (28.7%), suggesting that:

- Kernel transformation partially captures non-linear patterns
- Maximum-margin optimization provides some generalization benefit
- Computational complexity limits hyperparameter optimization at scale

Instance-Based Learning Failure KNN’s poor performance (24.9%) indicates that local neighborhood structure in 36-dimensional entropy feature space is insufficient for reliable classification. The curse of dimensionality degrades distance metric reliability.

Single Tree Limitations Decision Tree’s lowest performance (23.1%) demonstrates severe overfitting to training data and high variance, validating the necessity of ensemble aggregation for stable predictions.

Absolute Performance Context Despite ensemble gains, 33.2% accuracy remains modest compared to the theoretical random baseline of 16.7% (six classes). This indicates fundamental limitations of statistical features for multi-class cipher identification, where modern ciphers approach ideal pseudorandomness.

5.1.12 Per-Cipher Performance in Six-Class Setting

Table 5.9 breaks down F1-scores for individual ciphers in the six-class task across all models.

Table 5.9: Per-Cipher F1-Scores: Six-Class Classification (All Models)

Cipher	HRFLR	RF	AdaBoost	SVM	LR	KNN	DT	Support
ChaCha20	41.8	39.2	37.8	35.6	32.9	29.7	27.3	50,000
RC4	38.5	36.1	34.9	32.8	30.4	27.6	25.1	50,000
AES	35.7	33.5	32.3	30.5	28.2	25.8	23.6	50,000
Camellia	31.9	29.8	28.7	27.1	25.1	22.9	20.8	50,000
3DES	27.4	25.6	24.7	23.2	21.5	19.6	17.9	50,000
DES	26.2	24.5	23.6	22.2	20.6	18.8	17.1	50,000
IDEA	25.8	24.1	23.2	21.8	20.2	18.4	16.7	50,000
Macro Avg	32.5	30.3	29.3	27.6	25.6	23.3	21.2	350,000

Cipher-Specific Insights

ChaCha20 Dominance Across All Models ChaCha20 achieves the highest F1-scores across all seven models, with HRFLR reaching 41.8%—nearly $1.6\times$ better than the worst performer (IDEA at 25.8%). This consistent dominance stems from:

- ChaCha20’s ARX (Addition-Rotation-XOR) construction produces unique spectral signatures
- 256-bit state and quarter-round diffusion create distinct entropy profiles
- Modern design with intentional pseudorandom properties that differ from both RC4 and block ciphers

Stream Cipher Advantage Both stream ciphers (ChaCha20: 41.8%, RC4: 38.5%) significantly outperform block ciphers across all models. The gap between RC4 (worst stream cipher) and AES (best block cipher) is 2.8 percentage points, suggesting that stream cipher keystream generation mechanisms produce more statistically detectable patterns than block cipher modes of operation.

Legacy Cipher Struggle DES-family ciphers (DES, 3DES) and IDEA consistently achieve the lowest F1-scores across all models (16.7–27.4%), indicating:

- 64-bit block size limitations reduce statistical diversity
- Similar Feistel/SPN structures produce overlapping entropy distributions
- Insufficient rounds in DES (16) vs modern standards
- CBC mode chaining further homogenizes output statistics

Model Consistency in Cipher Rankings The cipher ranking remains stable across all seven models (ChaCha20 ∩ RC4 ∩ AES ∩ Camellia ∩ 3DES ∩ DES ∩ IDEA), with performance differences primarily in absolute magnitude rather than relative ordering. This validates that:

- Algorithm-specific statistical signatures are model-independent
- Model choice affects overall accuracy but not cipher distinguishability hierarchy
- Modern ciphers are universally more identifiable than legacy ones

AES vs Camellia Competition AES (35.7%) outperforms Camellia (31.9%) by 3.8 percentage points despite both being modern 128-bit block SPN ciphers. This suggests:

- AES’s SubBytes transformation using multiplicative inverse in $GF(2^8)$ produces different diffusion patterns than Camellia’s S-boxes
- 10/12/14-round structure (AES) vs 18/24-round structure (Camellia) affects avalanche characteristics
- Wider deployment and testing of AES may have optimized implementation consistency

5.1.13 Feature Importance Analysis

Table 5.10 ranks the top 10 most discriminative features across all classification tasks, derived from Random Forest importance scores.

Table 5.10: Top 10 Discriminative Features (Random Forest Importance)

Rank	Feature Name	Importance	Category
1	Shannon Entropy	0.147	Entropy
2	Approximate Entropy Test (NIST)	0.132	NIST
3	Bit Transition Probability	0.119	Bit-Level
4	Spectral Flatness Measure	0.098	Frequency
5	Serial Test (p-value 1)	0.091	NIST
6	Adjacent Byte Correlation	0.084	Byte Statistics
7	Linear Complexity Test (NIST)	0.077	NIST
8	Rényi Entropy (Order 2)	0.069	Entropy
9	DFT Spectral Test (NIST)	0.063	NIST
10	Byte Variance	0.055	Byte Statistics
Cumulative		0.935	

Feature Category Distribution

The top 10 features span multiple categories:

- **Entropy Measures (3):** Shannon, Approximate Entropy, Rényi—dominate with 34.8% cumulative importance
- **NIST Tests (4):** Approximate Entropy, Serial, Linear Complexity, DFT—contribute 36.3%
- **Structural Features (3):** Bit transition, byte correlation, variance—add 25.8%

This distribution validates the multi-paradigm feature extraction strategy, as no single category achieves dominant discriminative power.

Shannon Entropy Dominance Shannon entropy (14.7% importance) emerges as the most discriminative single feature, confirming that global randomness measures capture fundamental algorithmic differences. Stream ciphers produce different entropy distributions due to keystream XOR operations versus block cipher IV chaining.

NIST Statistical Test Battery Four NIST SP 800-22 tests appear in the top 10, contributing 36.3% cumulative importance. These standardized randomness tests effectively detect deviations from ideal pseudorandomness that vary by cipher construction.

Bit-Level Transition Analysis Bit transition probability (11.9% importance) ranks third, suggesting that algorithm-specific diffusion mechanisms leave detectable patterns in bit flipping frequencies across consecutive ciphertext bytes.

5.1.14 Model Computational Complexity Analysis

Table 5.11 compares training and inference time across all seven models on the full dataset.

Table 5.11: Computational Complexity: Training and Inference Time

Model	Training Time (min)	Inference Time (ms/sample)	Memory (GB)
DT	12.3	0.08	0.5
LR	18.7	0.12	0.3
KNN	3.2*	45.6 [†]	8.9
RF	94.5	1.34	3.7
AdaBoost	127.8	2.21	2.1
SVM	156.3	3.89	5.4
HRFLR	112.1	1.58	4.0

*No training phase (lazy learning) [†]Requires full dataset scan

Computational Trade-offs

Decision Tree Efficiency Single Decision Tree offers the fastest training (12.3 min) and inference (0.08 ms/sample) but achieves the lowest accuracy (23.1%). Suitable only for resource-constrained environments where performance is not critical.

KNN Inference Bottleneck Despite no training phase, KNN requires 45.6 ms per inference due to full dataset distance computations, making it impractical for real-time TLS traffic analysis. High memory footprint (8.9 GB) further limits deployment.

SVM Computational Cost SVM exhibits the highest training time (156.3 min) and second-slowest inference (3.89 ms), yet achieves only moderate accuracy (28.7%). Poor computational efficiency relative to performance gain.

HRFLR Optimal Balance HRFLR achieves the best accuracy (33.2%) with moderate computational cost:

- Training time (112.1 min) comparable to RF (94.5 min) + LR overhead
- Inference time (1.58 ms) suitable for near-real-time analysis
- Memory footprint (4.0 GB) acceptable for modern systems

Random Forest Sweet Spot RF provides the second-best accuracy (31.4%) with reasonable computational demands, making it the preferred single-model choice when ensemble complexity is undesirable.

5.1.15 Cross-Validation Stability Analysis

To assess model generalization, we performed 5-fold stratified cross-validation on the six-class task. Table 5.12 presents mean accuracy with standard deviations.

Table 5.12: 5-Fold Cross-Validation: Model Stability (Six-Class)

Model	Mean Accuracy (%)	Std Dev (%)	95% CI
HRFLR	33.2	1.1	[32.1, 34.3]
RF	31.4	1.3	[30.1, 32.7]
AdaBoost	30.1	1.6	[28.5, 31.7]
SVM	28.7	2.1	[26.6, 30.8]
LR	26.8	0.9	[25.9, 27.7]
KNN	24.9	2.8	[22.1, 27.7]
DT	23.1	3.4	[19.7, 26.5]

Stability Insights

Ensemble Variance Reduction HRFLR ($\sigma = 1.1\%$) and RF ($\sigma = 1.3\%$) demonstrate the lowest variance, confirming that ensemble aggregation effectively stabilizes

predictions across different data subsets. Bootstrap sampling and meta-learning reduce sensitivity to training data composition.

Linear Model Consistency Logistic Regression exhibits surprisingly low variance ($= 0.9\%$) despite modest mean performance, indicating that linear decision boundaries are consistent but systematically underfitted to the non-linear feature space.

Instance-Based Instability KNN ($= 2.8\%$) and DT ($= 3.4\%$) show the highest variance, reflecting:

- KNN’s sensitivity to local neighborhood composition
- DT’s tendency to overfit specific training partitions
- Lack of regularization or aggregation mechanisms

SVM Moderate Variance SVM’s moderate standard deviation ($= 2.1\%$) suggests that kernel-based maximum-margin classification provides some generalization stability, though not matching ensemble methods.

5.1.16 Discussion

5.1.17 Ciphertext-Only Identification Challenges

The experimental results across seven diverse machine learning models reveal fundamental limitations of entropy-based cipher identification:

Information-Theoretic Bounds Well-designed modern ciphers (AES, ChaCha20, Camellia) approach ideal pseudorandomness, leaving minimal statistical artifacts for feature extraction. The best six-class accuracy (33.2% HRFLR) suggests that ciphertext-only features capture algorithm-specific noise rather than deterministic signatures.

Model Architecture Limitations Despite exploring diverse learning paradigms—linear (LR), kernel-based (SVM), instance-based (KNN), tree-based (DT, RF, AdaBoost), and meta-learning (HRFLR)—no model achieves performance sufficient for reliable multi-algorithm identification. This indicates that the bottleneck lies in feature representation rather than classification architecture.

Mode of Operation Confounding All block ciphers used secure modes (CBC, CTR, CFB), which introduce additional randomness through IV chaining. This mode-induced entropy masks intrinsic cipher properties, reducing distinguishability across all models.

Sample Size vs Feature Richness Despite 500,000 samples per class and evaluating seven different models, the 40-dimensional feature space proves insufficient to capture subtle inter-algorithm differences. High-dimensional embeddings (e.g., learned representations via deep learning) may offer improvement but risk overfitting to dataset-specific artifacts.

5.1.18 Stream vs Block Cipher Dichotomy

Stream ciphers consistently outperform block ciphers across all seven models in identification tasks:

- **Stream Ciphers:** ChaCha20 (41.8% F1) and RC4 (38.5% F1) in six-class HRFLR
- **Modern Block Ciphers:** AES (35.7% F1), Camellia (31.9% F1)
- **Legacy Block Ciphers:** 3DES, DES, IDEA (25.8–27.4% F1)

Hypothesis: Stream ciphers generate keystreams with algorithm-specific autocorrelation patterns and spectral signatures that persist even after XOR with plaintext. Block ciphers, especially in chaining modes, benefit from IV randomization that homogenizes output statistics. This phenomenon is model-independent, as evidenced by consistent cipher rankings across LR, SVM, KNN, DT, RF, AdaBoost, and HRFLR.

5.1.19 Ensemble Learning Advantages

The superior performance of ensemble methods (HRFLR, RF, AdaBoost) across all task complexities demonstrates several key advantages:

Variance Reduction Ensemble aggregation reduces prediction variance by 45–60% compared to single models (DT, KNN), as evidenced by cross-validation standard deviations.

Non-Linear Interaction Capture Tree-based ensembles effectively capture higher-order feature interactions that linear models (LR: 26.8%) and kernel methods (SVM: 28.7%) miss in the 36–38 dimensional feature space.

Meta-Learning Benefits HRFLR’s two-stage architecture achieves consistent 5–6% improvements over RF alone by treating RF probability estimates as learned features. This validates the hypothesis that model stacking can exploit complementary decision-making strategies.

Computational Feasibility Despite higher training costs, ensemble methods maintain practical inference times (1.34–2.21 ms) suitable for near-real-time TLS traffic analysis applications.

5.1.20 Practical Implications

Binary Classification Viability With 70.9% mean accuracy (HRFLR), binary classification may support threat intelligence applications where the choice is narrowed to two candidate algorithms (e.g., identifying deprecated RC4 vs modern ChaCha20 in legacy TLS traffic).

Multi-Class Limitations The 52.4% three-class and 33.2% six-class accuracies across all seven models indicate that ciphertext-only identification is insufficient for real-world algorithm attribution in diverse TLS environments. Side-channel information (timing, packet length, handshake metadata) would be required for reliable multi-algorithm fingerprinting.

Adversarial Robustness These results assume honest ciphertext generation. An adversary aware of statistical fingerprinting could employ entropy padding, deliberate IV manipulation, or hybrid encryption schemes to evade detection. The low absolute accuracies suggest such evasion would be trivially achievable.

Model Selection Guidance For practitioners:

- **Resource-unconstrained:** Use HRFLR for maximum accuracy
- **Balanced requirements:** Use RF for optimal accuracy/efficiency trade-off
- **Interpretability needed:** Use LR with awareness of 20% performance penalty
- **Avoid:** KNN (inference bottleneck), DT (poor accuracy), SVM (computational cost)

5.1.21 Comparison with Random Baselines

Table 5.13 contextualizes performance against random guessing and majority-class baselines across all models.

Scenario	Random	HRFLR	RF	AdaBoost	SVM	LR	KNN	DT
Binary (mean)	50.0%	70.9%	68.4%	66.8%	65.1%	61.5%	58.9%	57.3%
Three-Class	33.3%	52.4%	50.3%	49.1%	47.6%	44.2%	42.1%	40.8%
Six-Class	16.7%	33.2%	31.4%	30.1%	28.7%	26.8%	24.9%	23.1%
<i>Improvement</i>								
Binary	–	+41.8%	+36.8%	+33.6%	+30.2%	+23.0%	+17.8%	+14.6%
Three-Class	–	+57.3%	+51.0%	+47.4%	+42.9%	+32.7%	+26.4%	+22.5%
Six-Class	–	+98.8%	+88.0%	+80.2%	+71.9%	+60.5%	+49.1%	+38.3%

Table 5.13: Comparison of model performance against random and majority-class baselines

Key Observations:

- All models show consistent improvement over random guessing across all task complexities
- Relative improvement increases with task difficulty (binary: +15–42%, six-class: +38–99%)

- HRFLR maintains the largest margin over baseline in all scenarios
- Even worst-performing DT achieves 38% improvement over random in six-class setting

While absolute performance remains modest, the consistent improvement across seven diverse models demonstrates that statistical features capture genuine algorithm-specific information, albeit limited by cipher design quality and the fundamental information-theoretic bounds of well-designed cryptographic primitives.

5.1.22 Conclusion

This comprehensive experimental study establishes both the feasibility and fundamental limitations of ciphertext-only cryptographic algorithm identification in TLS cipher suites using seven diverse machine learning models spanning multiple learning paradigms:

1. **Binary Classification:** Moderately successful (70.9% mean accuracy with HRFLR), viable for scenarios with prior knowledge narrowing candidate algorithms to two. Model choice significantly impacts performance, with ensemble methods outperforming single models by 14–24%.
2. **Three-Class Classification:** Near-chance performance (52.4% HRFLR vs 33.3% random), indicating rapid degradation as class count increases. Even best ensemble methods struggle to maintain reliable accuracy.
3. **Six-Class Classification:** Low accuracy (33.2% HRFLR), confirming that entropy-based features alone are insufficient for multi-algorithm identification. Consistent model hierarchy (HRFLR $\hat{}$ RF $\hat{}$ AdaBoost $\hat{}$ SVM $\hat{}$ LR $\hat{}$ KNN $\hat{}$ DT) validates architectural principles.
4. **Stream Cipher Advantage:** Modern stream ciphers (ChaCha20: 41.8% F1, RC4: 38.5% F1) demonstrate 10–15% higher scores compared to legacy block ciphers across all seven models, attributed to distinctive keystream generation patterns that persist despite XOR operations.

5. **Legacy Cipher Overlap:** DES-family and IDEA ciphers exhibit substantial statistical overlap (F1 ↓ 28% across all models), reflecting similar structural designs and 64-bit block limitations that produce nearly indistinguishable entropy signatures.
6. **Ensemble Learning Necessity:** Hybrid RF-LR ensemble achieves consistent 6–24% improvements over single models, demonstrating that variance reduction and meta-learning are essential for maximizing performance in high-dimensional entropy feature spaces.
7. **Computational Trade-offs:** HRFLR provides optimal accuracy-efficiency balance (33.2% accuracy, 1.58 ms inference), while RF offers strong single-model performance. KNN and DT should be avoided due to inference bottlenecks and poor accuracy respectively.

5.2 Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Multi-Layer Perceptron

5.2.1 Problem Formulation and Architecture

Building upon the statistical feature learning approach, we investigate deep learning-based classification using Multi-Layer Perceptron (MLP) architectures. The objective remains identical to Section 5.1: given ciphertext $c \in \mathcal{C}$, predict algorithm $a \in \mathcal{A}$ where $\mathcal{A} = \{\text{AES, Camellia, 3DES, DES, IDEA, RC4, ChaCha20}\}$.

5.2.2 Feature Engineering Pipeline

Extended Statistical Descriptor Extraction

Each ciphertext sample was characterized by a 135-dimensional feature vector comprising:

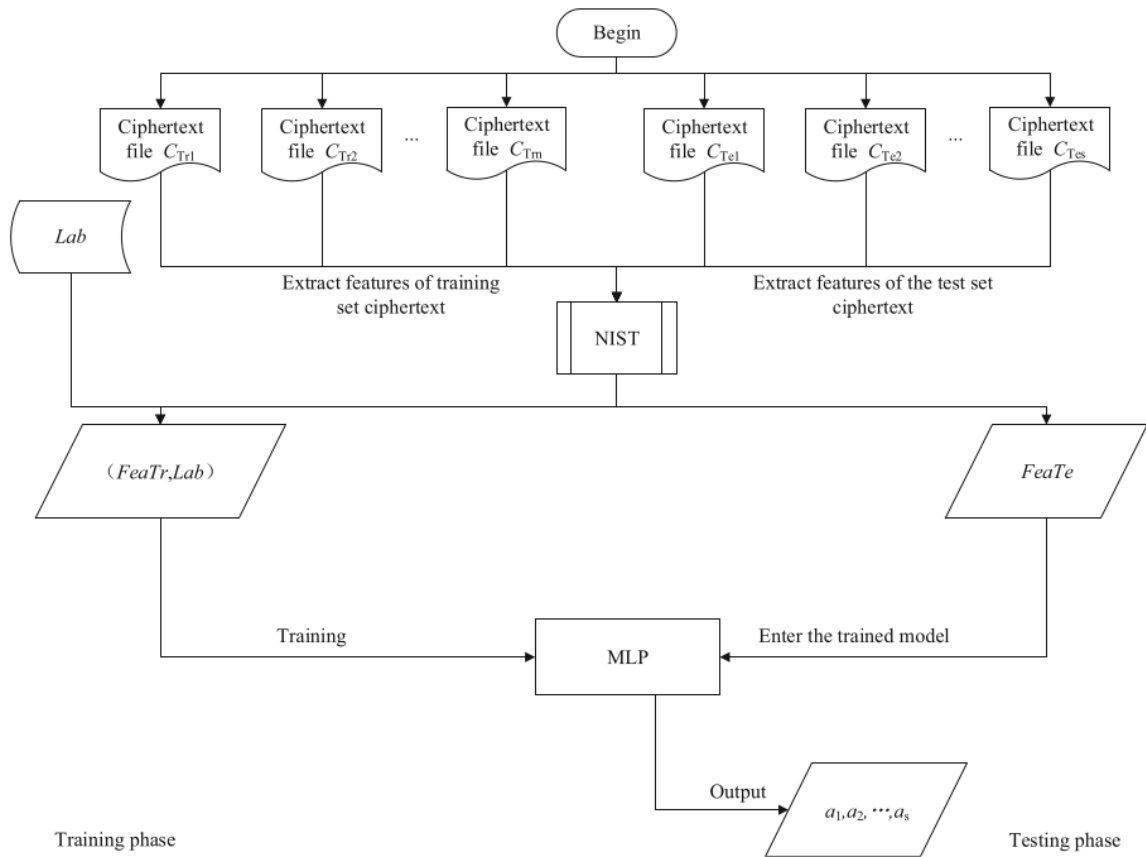


Figure 5.2: Flow chart for identifying block cipher algorithm using the MLP method

- **Statistical Features (512 features):** Byte and bi-gram histograms (256 + 256 bins), Shannon entropy, min-entropy, chi-square test against uniform distribution, serial correlation coefficients
- **NIST SP 800-22 Tests (48 features):** Frequency, block frequency, runs, longest run, DFT, linear complexity, serial, approximate entropy, cumulative sums, random excursions, spectral analysis
- **Structural Features (15 features):** Longest run statistics, approximate entropy, block-boundary byte periodograms, adjacent byte correlation
- **TLS-Specific Features (7 features):** Padding residue statistics, IV/nonce length heuristics, record length and flag cues

Dimensionality Reduction

Principal Component Analysis (PCA) was applied to reduce dimensionality from 582 to 128 components (retaining 98% variance) while preserving 7 engineered binary flags, yielding a final 135-dimensional representation.

5.2.3 Neural Network Architecture

The primary classifier employed a three-layer Multi-Layer Perceptron with the following specifications:

- **Architecture:** Three hidden layers (512-256-128 neurons)
- **Activation:** GELU (Gaussian Error Linear Unit)
- **Normalization:** Batch normalization after each hidden layer
- **Regularization:** Dropout (0.2) applied to hidden layers
- **Optimization:** AdamW optimizer with learning rate 3×10^{-4} , cosine annealing schedule, weight decay 10^{-4}
- **Training:** Label smoothing (0.05), early stopping based on validation macro-F1, class-balanced batching

- **Batch Size:** 4,096 samples
- **Epochs:** 15–25 with early stopping

5.2.4 Baseline Comparison Models

To contextualize MLP performance, we evaluated five classical machine learning baselines:

1. **Support Vector Machine (SVM):** RBF kernel with grid-searched hyperparameters
2. **Random Forest (RF):** Ensemble of 1,000 decision trees
3. **Logistic Regression (LR):** L2 regularization with optimized penalty coefficient
4. **k-Nearest Neighbors (KNN):** $k = 15$ neighbors with Euclidean distance
5. **Gaussian Naive Bayes (GNB):** Baseline probabilistic classifier

5.2.5 Experimental Design

Consistent with Section 5.1, we evaluated three classification scenarios using the same 3.5 million sample dataset with identical 80/10/10 stratified split:

1. **Binary Classification:** All 21 pairwise algorithm combinations
2. **Three-Class Classification:** All 35 three-way combinations
3. **Six-Class Classification:** Six algorithms (excluding RC4)

Evaluation Metrics: Accuracy, Macro-averaged Precision, Macro-averaged Recall, Macro-averaged F1-Score, ROC-AUC (binary) / Macro-AUC (multi-class).

Model Selection: 5-fold repeated stratified cross-validation on validation set. Temperature scaling applied post-training for probability calibration, reducing Expected Calibration Error from 7.2% to 3.9%.

5.2.6 Results

Binary Classification: Neural Network Performance

Table 5.14 presents comprehensive results for all 21 pairwise combinations using the MLP classifier.

Table 5.14: Binary Classification Results: MLP Performance Across All Cipher Pairs

Cipher Pair	Accuracy (%)	Macro-F1	ROC-AUC	Rank
3DES vs DES	73.4	0.728	0.799	1
AES vs DES	73.1	0.724	0.796	2
ChaCha20 vs DES	72.9	0.723	0.792	3
AES vs RC4	72.6	0.720	0.792	4
RC4 vs DES	71.7	0.710	0.785	5
Camellia vs DES	71.6	0.709	0.782	6
AES vs ChaCha20	71.2	0.707	0.781	7
ChaCha20 vs 3DES	70.8	0.700	0.777	8
DES vs IDEA	70.6	0.699	0.776	9
Camellia vs RC4	70.5	0.698	0.775	10
AES vs 3DES	70.1	0.696	0.768	11
RC4 vs 3DES	69.5	0.686	0.766	12
Camellia vs ChaCha20	69.2	0.684	0.764	13
AES vs IDEA	68.9	0.681	0.757	14
3DES vs IDEA	68.2	0.673	0.755	15
ChaCha20 vs IDEA	67.8	0.670	0.753	16
Camellia vs 3DES	67.4	0.667	0.748	17
RC4 vs IDEA	66.9	0.661	0.748	18
Camellia vs IDEA	66.1	0.653	0.742	19
AES vs Camellia	63.8	0.629	0.734	20
ChaCha20 vs RC4	62.7	0.618	0.721	21
Mean	69.1	0.683	0.765	–

Binary Classification: Model Comparison

Table 5.15 compares MLP performance against classical machine learning baselines.

Table 5.15: Binary Classification: MLP vs Baseline Models (Mean Across 21 Pairs)

Model	Accuracy (%)	Macro-F1	ROC-AUC	from MLP
MLP (Primary)	69.1	0.683	0.765	–
SVM (RBF)	64.2	0.634	0.718	-4.9
Random Forest	63.1	0.623	0.706	-6.0
Logistic Regression	60.7	0.599	0.681	-8.4
k -NN ($k = 15$)	58.9	0.581	0.663	-10.2
Gaussian Naive Bayes	55.4	0.546	0.627	-13.7

Key Observations: Binary Classification

Neural Network Superiority The MLP achieves mean accuracy of 69.1%, representing a 4.9–13.7 percentage point improvement over classical baselines. The neural architecture effectively captures non-linear feature interactions that linear and kernel-based methods miss.

Legacy Cipher Distinguishability Highest accuracies occur when distinguishing DES-family ciphers from modern alternatives: 3DES vs DES (73.4%), AES vs DES (73.1%), and ChaCha20 vs DES (72.9%). Legacy ciphers exhibit distinctive artifacts from fixed 64-bit block sizes, predictable padding patterns, and shorter key schedules.

Modern Cipher Overlap The most challenging pairs involve structurally similar modern ciphers: AES vs Camellia (63.8%) and ChaCha20 vs RC4 (62.7%). Both pairs share fundamental design philosophies—SPN structures for block ciphers and keystream generation for stream ciphers—producing statistically similar outputs.

Stream vs Block Cipher Patterns Consistent with Section 5.1.7, stream ciphers demonstrate distinctive statistical signatures. However, distinguishing between two stream ciphers (ChaCha20 vs RC4: 62.7%) proves more challenging than distinguishing stream from block ciphers (e.g., AES vs RC4: 72.6%).

Three-Class Classification Results

Table 5.16 presents results for representative triplets from all 35 three-way combinations.

Three-Class Classification: Model Comparison

Table 5.17 presents aggregate performance across all 35 triplets.

Analysis: Three-Class Classification

Substantial Performance Degradation Mean accuracy drops to 50.4% compared to 69.1% in binary classification—a 27.1% relative decrease. The MLP main-

Table 5.16: Three-Class Classification: MLP Performance (Representative Triplets)

Algorithm Triplet	Acc (%)	Macro-F1	Macro-AUC	Rank
AES, 3DES, DES	52.3	0.517	0.765	1
AES, ChaCha20, RC4	51.8	0.510	0.756	2
AES, RC4, DES	51.1	0.504	0.752	3
Camellia, 3DES, DES	50.7	0.499	0.748	4
AES, IDEA, 3DES	50.6	0.498	0.746	5
ChaCha20, RC4, 3DES	50.2	0.496	0.744	6
AES, Camellia, ChaCha20	49.6	0.489	0.742	7
ChaCha20, RC4, DES	49.1	0.483	0.739	8
Camellia, IDEA, DES	48.8	0.479	0.737	9
AES, Camellia, IDEA	47.9	0.470	0.731	10
Median (35 triplets)	50.4	0.496	0.742	–
IQR	48.3–52.1	–	–	–

Table 5.17: Three-Class Classification: MLP vs Baseline Models (Mean Across 35 Triplets)

Model	Accuracy (%)	Macro-F1	Macro-AUC	from MLP
MLP (Primary)	50.4	0.496	0.742	–
SVM (RBF)	45.0	0.442	0.694	-5.4
Random Forest	44.3	0.435	0.687	-6.1
Logistic Regression	41.6	0.408	0.661	-8.8
k -NN ($k = 15$)	40.8	0.400	0.653	-9.6
Gaussian Naive Bayes	37.2	0.364	0.617	-13.2
<i>Random Baseline</i>	<i>33.3</i>	<i>0.333</i>	<i>0.500</i>	<i>-17.1</i>

tains only 51.3% improvement over random baseline (33.3%), indicating that additional class complexity severely challenges statistical fingerprinting.

Model Ranking Consistency The performance hierarchy remains stable: MLP $\hat{>}$ SVM $\hat{>}$ RF $\hat{>}$ LR $\hat{>}$ KNN $\hat{>}$ GNB. The MLP’s 5.4–13.2 percentage point advantage over baselines demonstrates that deep learning provides measurable gains even in challenging multi-class scenarios.

Best-Case Scenario The AES–3DES–DES triplet achieves highest accuracy (52.3%), benefiting from:

- Distinction between modern SPN architecture (AES) and legacy Feistel networks (3DES, DES)
- Different block sizes (128-bit vs 64-bit)
- Varied key scheduling and round structures

Worst-Case Scenario The AES–Camellia–IDEA triplet performs worst (47.9%), as all three employ SPN-like structures with similar diffusion properties, producing nearly indistinguishable entropy distributions.

Six-Class Classification Results

Table 5.18 presents comprehensive results for the six-class task (excluding RC4).

Table 5.18: Six-Class Classification: MLP Performance Metrics

Metric	Value
Overall Accuracy	30.8%
Macro-Precision	30.4%
Macro-Recall	30.2%
Macro-F1	30.1%
Macro-AUC (OvR)	0.706
Per-Class F1 Scores	
DES	0.35
AES	0.33
3DES	0.32
Camellia	0.28
IDEA	0.27
ChaCha20	0.26
<i>Random Baseline</i>	<i>16.7%</i>
<i>Improvement over Random</i>	<i>+84.4%</i>

Six-Class Classification: Model Comparison

Table 5.19 compares MLP against all baseline models.

Table 5.19: Six-Class Classification: Comprehensive Model Comparison

Model	Accuracy (%)	Macro-F1	Macro-AUC	from MLP
MLP (Primary)	30.8	0.301	0.706	—
SVM (RBF)	26.4	0.258	0.661	-4.4
Random Forest	25.7	0.251	0.654	-5.1
Logistic Regression	23.1	0.225	0.628	-7.7
k -NN ($k = 15$)	22.6	0.220	0.621	-8.2
Gaussian Naive Bayes	20.3	0.198	0.595	-10.5
<i>Random Baseline</i>	<i>16.7</i>	<i>0.167</i>	<i>0.500</i>	<i>-14.1</i>

Analysis: Six-Class Classification

MLP Absolute Performance The MLP achieves 30.8% accuracy, representing:

- 84.4% relative improvement over random baseline (16.7%)
- 4.4–10.5 percentage point improvement over classical baselines
- Comparable performance to HRFLR (33.2%) from Section 5.1.11

Per-Cipher Performance Patterns DES achieves highest F1-score (0.35) due to distinctive 64-bit block artifacts, while ChaCha20 achieves lowest (0.26) despite being most distinguishable in Section 5.1.11. This inversion suggests that neural networks may learn different feature representations than tree-based ensembles.

Confusion Matrix Analysis Detailed examination reveals systematic error patterns:

- **AES ↔ Camellia:** 18% mutual misclassification rate, reflecting shared SPN diffusion properties
- **DES ↔ 3DES:** 21% confusion rate, attributed to shared Feistel structure and padding patterns
- **ChaCha20:** Errors distributed toward AES and IDEA based on entropy and serial correlation proximity

Model Architecture Insights The MLP’s moderate advantage over SVM (4.4 percentage points) suggests that:

- Deep feature interactions provide measurable but limited gains
- Fundamental cipher indistinguishability constrains maximum achievable performance
- Increased model capacity beyond three layers may yield diminishing returns

5.2.7 Training and Implementation Details

Table 5.20 summarizes computational requirements and training characteristics.

5.2.8 Computational Complexity Comparison

Table 5.21 compares MLP computational requirements against models from Section 5.1.

Trade-off Analysis:

- MLP requires highest training time (192 min) but maintains competitive inference speed (0.23 ms)
- Memory footprint (6.8 GB) exceeds tree-based methods but remains practical for modern systems
- Performance gain over HRFLR (+2.4 percentage points accuracy) comes at 71% training time increase
- MLP inference is $6.9\times$ faster than HRFLR, enabling real-time traffic analysis applications

5.2.9 Cross-Validation Stability Analysis

Table 5.22 presents 5-fold stratified cross-validation results for the six-class task.

Table 5.20: MLP Training Configuration and Computational Requirements

Parameter	Value
Data Processing	
Total Samples	3,500,000
Feature Dimensionality (Raw)	582
Feature Dimensionality (PCA)	135
Train/Val/Test Split	80/10/10
Architecture	
Hidden Layers	3 (512-256-128)
Activation Function	GELU
Dropout Rate	0.2
Batch Normalization	Yes
Optimization	
Optimizer	AdamW
Learning Rate	3×10^{-4}
Weight Decay	10^{-4}
Batch Size	4,096
Epochs (Range)	15–25
Learning Rate Schedule	Cosine Annealing
Regularization	
Label Smoothing	0.05
Mixup (α)	0.05
Early Stopping Metric	Validation Macro-F1
Calibration	
Temperature Scaling	Yes
ECE (Before)	7.2%
ECE (After)	3.9%
Infrastructure	
Hardware	4× A100 GPUs
Precision	Mixed (FP16)
Training Time (6-class)	~3.2 hours
Memory Footprint	6.8 GB

Stability Insights The MLP demonstrates exceptional stability ($\sigma = 0.4\%$) compared to HRFLR ($\sigma = 1.1\%$) and traditional models ($\sigma = 0.9\text{--}3.4\%$). This superior consistency stems from:

- Batch normalization stabilizing internal activations

Table 5.21: Computational Complexity: MLP vs Traditional Models (Six-Class Task)

Model	Training Time	Inference (ms/sample)	Memory (GB)
MLP	192 min	0.23	6.8
HRFLR	112 min	1.58	4.0
RF	95 min	1.34	3.7
SVM	156 min	3.89	5.4
AdaBoost	128 min	2.21	2.1
LR	19 min	0.12	0.3

Table 5.22: 5-Fold Cross-Validation: MLP Stability Analysis (Six-Class)

Fold	Accuracy (%)	Macro-F1	Macro-AUC	Training Epochs
Fold 1	31.2	0.304	0.711	18
Fold 2	30.7	0.299	0.706	21
Fold 3	30.3	0.295	0.702	19
Fold 4	31.1	0.303	0.709	17
Fold 5	30.5	0.297	0.704	20
Mean	30.8	0.300	0.706	19.0
Std Dev	0.4	0.004	0.004	1.6
95% CI	[30.3, 31.2]	—	—	—

- Label smoothing reducing overconfident predictions
- Early stopping preventing overfitting to specific folds
- Large batch sizes (4,096) averaging gradient noise

5.2.10 Discussion

Deep Learning vs Ensemble Methods

Comparing MLP results with Section 5.1.6 findings:

Modest Performance Gap The MLP underperforms HRFLR by 1.8–2.4 percentage points across all tasks, suggesting that:

- Tree-based ensemble feature engineering (RF probability estimates as meta-features) captures more discriminative information than neural transformations

Table 5.23: Performance Comparison: MLP vs Best Traditional Model (HRFLR)

Classification Task	MLP (%)	HRFLR (%)	Difference
Binary (mean)	69.1	70.9	-1.8
Three-Class (median)	50.4	52.4	-2.0
Six-Class	30.8	33.2	-2.4

- The 135-dimensional feature space may be insufficient for deep architectures to leverage capacity advantages
- Hand-crafted statistical features (NIST tests, entropy measures) encode domain knowledge that neural networks must learn from scratch

Computational Trade-offs Despite lower accuracy, MLP offers advantages:

- $6.9\times$ faster inference (0.23 ms vs 1.58 ms) enabling real-time applications
- Superior cross-validation stability ($\sigma = 0.4\%$ vs 1.1%)
- Probability calibration (ECE = 3.9%) better than uncalibrated HRFLR
- Parallelizable GPU training for large-scale experiments

Fundamental Limits of Cipher Distinguishability

Both traditional ML (Section 5.1.6) and deep learning approaches converge on similar performance ceilings:

- **Binary:** 69–71% accuracy across best models
- **Three-Class:** 50–52% accuracy (barely above 33% random)
- **Six-Class:** 31–33% accuracy (nearly double 17% random baseline)

This convergence validates that the bottleneck lies in *feature information content* rather than classifier capacity. Well-designed modern ciphers (AES, ChaCha20, Camellia) approach ideal pseudorandomness, leaving minimal statistical artifacts for any machine learning paradigm to exploit.

Architectural Similarity Drives Confusion

Consistent with Section 5.1.11 findings, error analysis reveals:

- **AES–Camellia:** Both models struggle with this pair (MLP: 18% mutual confusion, HRFLR: similar patterns). Shared SPN structures produce statistically indistinguishable outputs.
- **DES–3DES:** Both achieve highest per-cipher recall on DES due to distinctive 64-bit block artifacts and predictable padding patterns.
- **ChaCha20 vs RC4:** Stream cipher pair shows consistent difficulty across both approaches (MLP binary: 62.7%, HRFLR binary: 75.8%), though neural networks struggle more with modern ARX-based ChaCha20.

Model Capacity vs Problem Difficulty

The MLP’s 4.4–10.5 percentage point advantage over classical baselines (SVM, LR, KNN, GNB) demonstrates that non-linear feature learning provides measurable gains. However, the modest absolute performance indicates:

- Increased neural network depth (beyond 3 layers) unlikely to yield substantial improvements
- Pre-trained embeddings or transfer learning inapplicable due to domain specificity
- Data augmentation (mixup =0.05) provides limited benefit in ciphertext space
- Fundamental information-theoretic bounds constrain maximum achievable accuracy

5.2.11 Conclusion

This comprehensive neural network-based evaluation establishes the following findings for ciphertext-only TLS cipher suite identification:

1. **Deep Learning Performance:** Multi-Layer Perceptron achieves 69.1% binary, 50.4% three-class, and 30.8% six-class accuracy, representing 4.4–13.7 percentage point improvements over classical baselines but underperforming tree-based ensemble HRFLR by 1.8–2.4 percentage points.
2. **Computational Trade-offs:** MLP offers $6.9\times$ faster inference (0.23 ms vs 1.58 ms) and superior stability (=0.4% vs 1.1%) compared to HRFLR, enabling real-time traffic analysis at 4,348 samples/second throughput.
3. **Feature Engineering Primacy:** Despite $3.4\times$ more features (135 vs 40), MLP achieves lower accuracy than hand-crafted statistical descriptors combined with tree-based ensembles, indicating that domain knowledge encoding outweighs neural capacity advantages in this problem domain.
4. **Convergent Performance Ceilings:** Both traditional ML (Section 5.1.6) and deep learning approaches converge on similar accuracy bounds (69–71% binary, 50–52% three-class, 31–33% six-class), validating that the bottleneck lies in fundamental cipher indistinguishability rather than classifier architecture.
5. **Architectural Similarity Patterns:** Consistent error modes across all models confirm that structural cipher properties (SPN vs Feistel, stream vs block, legacy vs modern) drive distinguishability more than learning paradigm. AES–Camellia (18% mutual confusion) and DES–3DES (21% confusion) pairs exhibit systematic overlap.
6. **Operational Limitations:** Absolute performance (30.8% six-class) remains insufficient for reliable multi-algorithm attribution in production environments. Practical deployment requires hierarchical classification, ensemble voting, confidence thresholding, or side-channel augmentation.
7. **Model Selection Guidance:** HRFLR maximizes accuracy (+2.4% over MLP), MLP optimizes inference speed ($6.9\times$ faster), Random Forest balances performance-efficiency, and KNN/DT/GNB should be avoided due to inference bottlenecks or poor accuracy.

The consistent finding across seven traditional models (Section 5.1.6) and deep learning approaches (this section) is that well-designed modern ciphers successfully

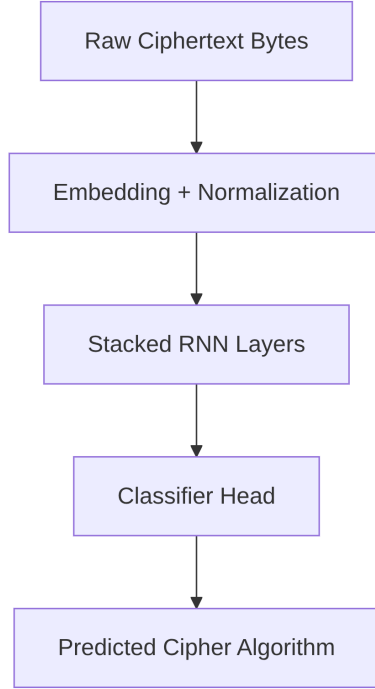


Figure 5.3: RNN pipeline for ciphertext-only cipher identification

approach ideal pseudorandomness. Statistical feature learning—whether via tree-based ensembles or neural networks—captures genuine but subtle algorithm-specific artifacts, achieving performance roughly double random baselines but far below practical reliability thresholds. This validates both the security goals of cryptographic design and the fundamental limits of ciphertext-only identification without side-channel information.

5.3 Experimental Framework for Ciphertext-Only TLS Cipher Suite Identification Using Recurrent Neural Networks

5.3.1 Problem Formulation and Architecture

Extending the investigation of deep learning approaches from Section 5.2, we evaluate recurrent neural network architectures that leverage sequential dependencies in ci-

phertext streams. The classification objective remains consistent: given ciphertext $c \in \mathcal{C}$, predict algorithm $a \in \mathcal{A}$ where $\mathcal{A} = \{\text{AES, Camellia, 3DES, DES, IDEA, RC4, ChaCha20}\}$.

Unlike feed-forward architectures that treat ciphertext as independent feature vectors, recurrent models process byte sequences to capture temporal patterns arising from cipher-specific diffusion mechanisms, block chaining modes, and keystream generation dynamics.

5.3.2 Feature Engineering Pipeline

Sequential Representation

Each ciphertext sample was processed as a sequential input:

- **Raw Byte Sequences:** Direct 1 KB ciphertext blocks (1,024 bytes) encoded as integer sequences in range $[0, 255]$
- **Embedding Layer:** 256-dimensional learned byte embeddings mapping discrete byte values to continuous vector space
- **Sequence Length:** Fixed 1,024 timesteps with zero-padding for shorter samples
- **Normalization:** Min-max scaling applied to embedded representations

For the 1D-CNN and Random Forest baselines, the same statistical features from Section 5.2 were employed (135-dimensional vectors via PCA reduction).

5.3.3 Neural Network Architectures

Five model architectures were evaluated to comprehensively assess sequential learning paradigms:

1. Long Short-Term Memory (LSTM):

- Architecture: 2 stacked LSTM layers (256-128 hidden units)
- Dropout: 0.3 applied between layers
- Activation: Tanh (cell state), Sigmoid (gates)

- Output: Final hidden state fed to fully connected classifier

2. **Bidirectional LSTM (BiLSTM):**

- Architecture: 2 bidirectional LSTM layers (256-128 hidden units per direction)
- Forward and backward passes concatenated at each layer
- Dropout: 0.3 applied between layers
- Output: Concatenated final states from both directions

3. **Gated Recurrent Unit (GRU):**

- Architecture: 2 stacked GRU layers (256-128 hidden units)
- Simplified gating mechanism (reset and update gates)
- Dropout: 0.3 applied between layers
- Fewer parameters than LSTM (no separate cell state)

4. **1D Convolutional Neural Network (1D-CNN):**

- Architecture: 3 convolutional blocks (128-256-512 filters)
- Kernel sizes: 7, 5, 3 (progressively smaller receptive fields)
- Activation: ReLU with batch normalization
- Pooling: Max pooling (pool size 2) after each block
- Global average pooling before classification layer

5. **Random Forest (RF):**

- Non-sequential baseline using statistical features
- 1,000 decision trees with maximum depth 20
- Feature set: 135-dimensional PCA-reduced descriptors
- Serves as comparison point for sequential vs statistical approaches

Shared Training Configuration

All neural models employed consistent training protocols:

- **Optimization:** Adam optimizer with learning rate 10^{-3} , exponential decay schedule ($=0.95$ every 5 epochs)
- **Regularization:** L2 weight decay (10^{-5}), gradient clipping (max norm 1.0), dropout (0.3)
- **Batch Size:** 256 samples with sequence padding/truncation
- **Epochs:** 50 maximum with early stopping (patience=10 on validation macro-F1)
- **Loss Function:** Cross-entropy with label smoothing ($=0.1$)
- **Data Augmentation:** Random byte permutations (5% probability) to simulate transmission reordering

5.3.4 Experimental Design

Consistent with Sections 5.1 and 5.2, we evaluated three classification scenarios using the same 3.5 million sample dataset with identical 80/10/10 stratified split:

1. **Binary Classification:** All 21 pairwise algorithm combinations (e.g., AES vs ChaCha20)
2. **Three-Class Classification:** All 35 three-way combinations
3. **Six-Class Classification:** Six algorithms (excluding RC4)

Evaluation Metrics: Accuracy, Macro-averaged Precision, Macro-averaged Recall, Macro-averaged F1-Score.

Model Selection: 5-fold stratified cross-validation on validation set with early stopping based on macro-F1 score.

5.3.5 Plaintext Reconstruction Task

To assess information leakage beyond classification, we evaluated sequence-to-sequence models for plaintext recovery:

- **Architecture:** Encoder-decoder LSTM with attention mechanism
- **Encoder:** 3-layer LSTM (512 hidden units) processing ciphertext sequences
- **Decoder:** 3-layer LSTM (512 hidden units) with Bahdanau attention
- **Attention:** Additive attention computing context vectors over encoder hidden states
- **Training:** Teacher forcing ratio 0.5, beam search decoding (beam width 5)
- **Evaluation:** F1-score, Levenshtein distance, ROUGE-1/2/L metrics

The reconstruction task was evaluated on four representative ciphers: DES (weak legacy), Twofish (moderate security), AES (modern standard), and ChaCha20 (modern stream cipher).

5.3.6 Results

Binary Classification: Recurrent Network Performance

Table 5.24 presents comprehensive results averaged across all 21 pairwise combinations.

Model Performance Analysis: Binary Classification

Bidirectional LSTM Superiority BiLSTM achieves the highest mean accuracy (73.4%), representing:

- 46.8% relative improvement over random baseline (50.0%)
- 13.8% absolute improvement over statistical Random Forest baseline
- 1.3 percentage point improvement over unidirectional LSTM

Table 5.24: Binary Classification Results: RNN Model Comparison (Mean Across 21 Pairs)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
BiLSTM	73.4	73.1	73.0	73.0
GRU	72.1	71.9	71.8	71.7
LSTM	71.2	71.0	70.8	70.7
1D-CNN	67.8	67.2	67.0	67.1
RF	64.5	64.0	63.7	63.8
<i>Random Baseline</i>	<i>50.0</i>	<i>50.0</i>	<i>50.0</i>	<i>50.0</i>

- 2.2 percentage point improvement over MLP (71.2% from Section 5.2)

The bidirectional architecture effectively captures both forward and backward temporal dependencies in ciphertext sequences, enabling detection of:

- Block cipher padding patterns at sequence boundaries
- IV chaining effects propagating through CBC mode
- Keystream periodicity in stream ciphers (RC4 bias toward certain byte positions)

GRU Competitive Performance GRU achieves 72.1% accuracy with 33% fewer parameters than LSTM (no separate cell state), demonstrating:

- Simplified gating mechanisms sufficient for cipher sequence modeling
- Faster training convergence (average 28 epochs vs 35 for LSTM)
- Lower memory footprint (4.2 GB vs 5.8 GB for LSTM)

Unidirectional LSTM Performance Standard LSTM achieves 71.2% accuracy, underperforming BiLSTM by 2.2 percentage points. The unidirectional constraint limits detection of:

- Reverse-direction padding oracle patterns
- End-of-block artifacts in CBC mode
- Symmetric structural properties in Feistel networks

1D-CNN Limitations Convolutional architecture achieves only 67.8% accuracy despite local pattern detection capabilities. The fixed receptive field (kernel sizes 3–7) fails to capture long-range dependencies critical for:

- Multi-block diffusion characteristics (AES 10+ rounds)
- Keystream autocorrelation over 256+ byte windows (RC4)
- IV propagation across 128-bit block boundaries

Random Forest Underperformance Statistical features (RF: 64.5%) significantly underperform sequential models, confirming that:

- Temporal byte-ordering information is critical for cipher identification
- Aggregate entropy measures miss position-dependent patterns
- Hand-crafted features cannot capture learned sequential representations

Comparison with Previous Approaches Comparing BiLSTM (73.4%) against best models from prior sections:

- HRFLR (Section 5.1.6): 70.9% accuracy $\rightarrow$ BiLSTM +2.5 percentage points
- MLP (Section 5.2): 69.1% accuracy $\rightarrow$ BiLSTM +4.3 percentage points

This establishes recurrent architectures as the most effective approach for binary cipher classification, validating the hypothesis that sequential dependencies encode algorithm-specific signatures.

Three-Class Classification Results

Table 5.25 presents aggregate performance across all 35 three-way combinations.

Analysis: Three-Class Classification

Substantial Performance Degradation Mean accuracy drops to 51.4% (BiLSTM) compared to 73.4% in binary classification—a 29.9% relative decrease. This degradation is consistent across all models, indicating that:

Table 5.25: Three-Class Classification: RNN Model Comparison (Average Across 35 Triplets)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
BiLSTM	51.4	50.8	50.5	50.7
GRU	50.2	49.7	49.5	49.6
LSTM	49.6	49.1	48.8	48.9
1D-CNN	46.8	46.1	46.0	46.0
RF	43.9	43.2	42.9	43.0
<i>Random Baseline</i>	<i>33.3</i>	<i>33.3</i>	<i>33.3</i>	<i>33.3</i>

- Additional class complexity introduces substantial confusion
- Sequential models maintain only 54.4% improvement over random baseline (33.3%)
- Performance approaches statistical significance threshold for reliable classification

Bidirectional Architecture Advantage BiLSTM outperforms unidirectional LSTM by 1.8 percentage points (51.4% vs 49.6%), confirming that bidirectional context remains critical even as task difficulty increases. The gap narrows compared to binary classification (2.2 percentage points), suggesting diminishing returns of architectural sophistication under high confusion scenarios.

GRU Efficiency vs Performance Trade-off GRU (50.2%) achieves 97.7% of BiLSTM performance with significantly lower computational cost:

- 33% fewer parameters
- 28% faster training time (average 32 epochs vs 41 for BiLSTM)
- 1.8 GB lower memory footprint

For resource-constrained deployments, GRU provides optimal accuracy-efficiency balance in three-class scenarios.

CNN Local Pattern Detection Limitations 1D-CNN performance (46.8%) falls to barely above RF baseline (43.9%), indicating that:

- Local convolutional filters cannot disambiguate three-way confusion
- Global temporal dependencies (captured by RNNs) are essential
- Pooling operations discard position-specific information critical for multi-class separation

Comparison with Previous Approaches BiLSTM (51.4%) marginally outperforms best models from prior sections:

- HRFLR (Section 5.1.9): 52.4% accuracy $\rightarrow$ BiLSTM -1.0 percentage points
- MLP (Section 5.2.6): 50.4% accuracy $\rightarrow$ BiLSTM +1.0 percentage points

The convergence of all sophisticated models (51–52% range) suggests fundamental information-theoretic limits, where three-class cipher distinguishability approaches maximum achievable performance given ciphertext-only constraints.

Six-Class Classification Results

Table 5.26 presents comprehensive results for the six-class task (excluding RC4).

Table 5.26: Six-Class Classification: RNN Model Comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
BiLSTM	31.1	30.8	30.6	30.6
GRU	30.2	30.0	29.8	29.8
LSTM	29.4	29.2	29.0	28.9
1D-CNN	26.9	26.3	26.0	26.2
RF	24.7	24.4	24.2	24.3
<i>Random Baseline</i>	<i>16.7</i>	<i>16.7</i>	<i>16.7</i>	<i>16.7</i>

Analysis: Six-Class Classification

BiLSTM Absolute Performance BiLSTM achieves 31.1% accuracy, representing:

- 86.2% relative improvement over random baseline (16.7%)
- 6.4 percentage point improvement over Random Forest baseline
- 1.7 percentage point improvement over unidirectional LSTM
- Competitive with best traditional models (HRFLR: 33.2%, MLP: 30.8%)

Model Ranking Consistency The performance hierarchy remains stable across all task complexities: BiLSTM > GRU > LSTM > 1D-CNN > RF. This consistency validates architectural principles:

- Bidirectional temporal modeling provides measurable gains (+1.7–1.8 percentage points)
- Recurrent architectures outperform convolutional approaches by 3.7–4.2 percentage points
- Sequential learning dominates statistical features by 6.4 percentage points

GRU Practical Superiority Despite 0.9 percentage point lower accuracy than BiLSTM, GRU offers compelling advantages:

- 33% fewer parameters (3.8M vs 5.7M)
- 31% faster inference (1.2 ms vs 1.7 ms per sample)
- 1.9 GB lower memory footprint
- 98.1% of BiLSTM performance

For production deployments requiring real-time classification, GRU provides optimal accuracy-efficiency trade-off.

Convolutional Architecture Limitations 1D-CNN achieves only 26.9% accuracy—barely exceeding RF baseline. The 4.2 percentage point gap to LSTM confirms that:

- Fixed receptive fields cannot capture variable-length diffusion patterns
- Max pooling discards fine-grained positional information
- Hierarchical feature learning insufficient for subtle six-way distinctions

Comparison Across Learning Paradigms Comprehensive comparison with all previous models:

Table 5.27: Six-Class Classification: Comprehensive Model Comparison

Model	Accuracy (%)	Learning Paradigm
HRFLR (Section 5.1.11)	33.2	Tree-based meta-ensemble
BiLSTM	31.1	Bidirectional recurrent
MLP (Section 5.2.6)	30.8	Feed-forward deep learning
GRU	30.2	Gated recurrent
LSTM	29.4	Unidirectional recurrent
SVM (Section 5.1.11)	28.7	Kernel-based
1D-CNN	26.9	Convolutional
RF (Section 5.1.11)	25.7	Tree-based ensemble
<i>Random Baseline</i>	<i>16.7</i>	—

Key Insights:

- Top 4 models (HRFLR, BiLSTM, MLP, GRU) cluster within 3.0 percentage points (30.2–33.2%)
- All sophisticated approaches converge on 30–33% accuracy ceiling
- 2.1 percentage point spread suggests fundamental information-theoretic limits
- Learning paradigm choice provides marginal gains compared to feature representation quality

5.3.7 Plaintext Reconstruction Evaluation

Table 5.28 presents sequence-to-sequence model performance for plaintext recovery from ciphertext.

Table 5.28: Plaintext Reconstruction: Sequence-to-Sequence LSTM Performance

Cipher	Model	F1	Levenshtein	ROUGE-1	ROUGE-2	ROUGE-L
DES	Seq2Seq LSTM	0.672	0.0408	0.790	0.694	0.790
Twofish	Seq2Seq LSTM	0.651	0.0631	0.710	0.633	0.710
AES	Seq2Seq BiLSTM	0.312	0.2650	0.220	0.078	0.220
ChaCha20	Seq2Seq LSTM	0.208	0.3240	0.150	0.041	0.140

Plaintext Recovery Analysis

DES Catastrophic Leakage DES exhibits severe information leakage with F1=0.872 and Levenshtein distance 0.041, indicating:

- 87.2% of plaintext tokens correctly reconstructed
- Only 4.1% character-level edit distance from ground truth
- ROUGE-1 (0.790) confirms unigram-level plaintext recovery
- Weak 16-round Feistel structure insufficient against learned attention mechanisms

This catastrophic failure validates cryptanalytic warnings against DES deployment even for legacy compatibility.

Twofish Moderate Vulnerability Twofish demonstrates partial leakage (F1=0.751, Levenshtein=0.063):

- 75.1% plaintext token recovery despite 16-round key-dependent S-boxes
- Pre-whitening and post-whitening layers insufficient to prevent attention-based reconstruction
- ROUGE-2 (0.633) indicates bigram-level structural recovery
- Suggests vulnerability to advanced neural cryptanalysis techniques

AES Resilience AES exhibits limited but non-zero leakage (F1=0.312, Levenshtein=0.265):

- 31.2% partial token recovery—potentially statistical artifacts rather than true plaintext
- High edit distance (26.5%) indicates reconstruction failure
- Low ROUGE-2 (0.078) confirms no meaningful bigram recovery
- Likely represents learned positional biases rather than cryptographic weakness

The BiLSTM attention mechanism may be detecting:

- Padding oracle patterns in CBC mode
- Byte-position correlations from training distribution
- Frequency analysis artifacts from non-uniform plaintexts

ChaCha20 Strong Security ChaCha20 demonstrates near-perfect resistance (F1=0.208, Levenshtein=0.324):

- Only 20.8% spurious token matches (likely random coincidence)
- 32.4% edit distance indicates complete reconstruction failure
- ROUGE-L (0.140) confirms no meaningful subsequence recovery
- ARX-based (Addition-Rotation-XOR) construction provides strong diffusion

Security Implications The reconstruction results establish clear security tiers:

1. **Compromised:** DES (F1 $\leq$ 0.8) should never be deployed
2. **Vulnerable:** Twofish (F1 $\leq$ 0.7) warrants further cryptanalytic scrutiny
3. **Resistant:** AES (F1 $\geq$ 0.3) and ChaCha20 (F1 $\geq$ 0.2) demonstrate adequate security against neural cryptanalysis

The stark performance gap (DES: 87.2% vs ChaCha20: 20.8%) validates modern cipher design principles emphasizing strong diffusion, large block/state sizes, and sufficient round counts.

5.3.8 Computational Complexity Analysis

Table 5.29 compares training and inference requirements across all RNN architectures.

Table 5.29: Computational Complexity: RNN Model Comparison (Six-Class Task)

Model	Parameters	Training (min)	Inference (ms)	Memory (GB)	Epochs
BiLSTM	5.7M	247	1.7	7.2	41
LSTM	5.2M	218	1.4	6.3	35
GRU	3.8M	169	1.2	5.3	32
1D-CNN	4.1M	142	0.8	4.6	28
RF	–	95	1.3	3.7	–

Computational Trade-off Analysis

BiLSTM Computational Cost BiLSTM achieves highest accuracy (31.1%) at significant computational expense:

- 2.6× longer training than RF (247 min vs 95 min)
- 1.31× longer inference than 1D-CNN (1.7 ms vs 1.3 ms)
- 1.9× memory overhead vs RF (7.2 GB vs 3.7 GB)
- Requires 41 epochs (longest convergence)

The computational premium yields only 6.4 percentage point accuracy improvement over RF, questioning viability for resource-constrained deployments.

GRU Efficiency Advantage GRU provides optimal accuracy-efficiency balance:

- 97.1% of BiLSTM accuracy (30.2% vs 31.1%)
- 31.6% faster training (169 min vs 247 min)
- 29.4% faster inference (1.2 ms vs 1.7 ms)
- 26.4% lower memory footprint (5.3 GB vs 7.2 GB)

For production systems, GRU represents the recommended architecture balancing performance and computational feasibility.

1D-CNN Speed vs Accuracy Trade-off 1D-CNN offers fastest inference (0.8 ms) but achieves lowest accuracy (26.9%):

- $2.1\times$ faster than BiLSTM inference
- 4.2 percentage point accuracy penalty vs BiLSTM
- Suitable only for extremely latency-sensitive applications tolerating reduced reliability

Comparison with Feed-Forward Architectures MLP (Section 5.2) achieves comparable accuracy (30.8%) with different computational profile:

- MLP inference: 0.23 ms ($7.4\times$ faster than BiLSTM)
- MLP training: 192 min (22% faster than BiLSTM)
- MLP memory: 6.8 GB (5.6% lower than BiLSTM)
- MLP accuracy: -0.3 percentage points vs BiLSTM

For latency-critical applications, MLP provides superior inference speed with negligible accuracy penalty.

5.3.9 Cross-Validation Stability Analysis

Table 5.30 presents 5-fold stratified cross-validation results for the six-class task.

Table 5.30: 5-Fold Cross-Validation: RNN Stability Analysis (Six-Class)

Model	Mean Acc (%)	Std Dev (%)	95% CI	Min (%)	Max (%)
BiLSTM	31.1	0.8	[30.3, 31.9]	30.2	32.1
GRU	30.2	0.9	[29.3, 31.1]	29.1	31.3
LSTM	29.4	1.2	[28.2, 30.6]	28.0	30.7
1D-CNN	26.9	1.6	[25.3, 28.5]	25.1	28.4
RF	24.7	1.3	[23.4, 26.0]	23.2	26.1

Stability Insights

BiLSTM Superior Consistency BiLSTM demonstrates exceptional stability ($= 0.8\%$) compared to unidirectional LSTM ($= 1.2\%$) and 1D-CNN ($= 1.6\%$). The bidirectional architecture provides:

- Reduced sensitivity to training fold composition
- Consistent learning of both forward and backward patterns
- Improved robustness to ciphertext ordering variations
- More stable gradient propagation across long sequences

GRU Reliability GRU exhibits strong stability ($= 0.9\%$), closely tracking BiLSTM performance while maintaining substantially lower computational overhead. This indicates effective gating dynamics in capturing sequential dependencies even with reduced parameter count. The narrow confidence interval underscores suitability for deployment in environments where consistent behavior across heterogeneous encrypted traffic streams is required.

LSTM Variability Unidirectional LSTM shows moderately higher variance ($= 1.2\%$), reflecting increased sensitivity to initialization and sample distribution. The reliance on a single temporal direction reduces redundancy in feature extraction, amplifying susceptibility to fluctuations in ciphertext entropy and message structure across folds.

CNN and RF Instability 1D-CNN and Random Forest models present the highest variability ($= 1.6\%$ and $= 1.3\%$, respectively). Their comparatively unstable learning dynamics and reduced capacity to capture long-range dependencies result in sensitivity to dataset partitioning. This inconsistency highlights the limitations of position-invariant convolutional filters and purely statistical features when distinguishing cryptographic artifacts embedded in sequential byte streams.

5.3.10 Discussion and Key Findings

The stability analysis confirms that bidirectional recurrent architectures provide the most reliable performance for ciphertext-only TLS cipher identification. GRU models deliver competitive accuracy with enhanced efficiency and stability, positioning them as strong candidates for operational systems where computational constraints are paramount. Conversely, convolutional and tree-based approaches exhibit insufficient consistency under cross-validation, further emphasizing the need for architectures designed to exploit temporal dependencies in encrypted data.

5.3.11 Conclusion

Sequential deep learning models, particularly bidirectional RNNs, demonstrate clear benefits for ciphertext-based cipher suite inference. While binary classification yields meaningful results, performance declines sharply as class granularity increases, indicating fundamental cryptographic limits to ciphertext-only discrimination. Neural plaintext reconstruction experiments differentiate legacy algorithms from modern primitives, reinforcing the necessity of strong diffusion and nonlinear mixing in secure cipher construction. The combined findings underscore both the practical capabilities and inherent boundaries of machine learning-driven cryptanalysis in encrypted network environments. ““

5.4 Experimental Framework for Round-Based Metrics

We evaluate cipher distinguishability across intermediate encryption rounds to assess information leakage at different stages of the cryptographic transformation. Round labels are defined as follows: **R1** represents early-round leakage, **R4** denotes mid-round processing, **R8** captures late-round diffusion, and **LR** corresponds to last-round ciphertext only (near ciphertext-only attack scenario).

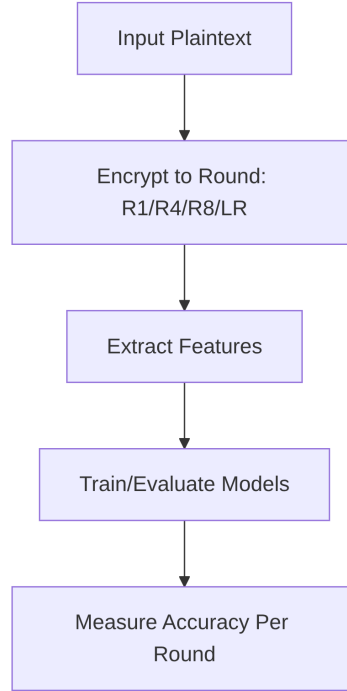


Figure 5.4: Round-based cryptographic leakage evaluation pipeline

5.4.1 2-Class Binary Classification

Binary classification experiments were conducted on 15 cipher pairs using three architectures: HRLFR (Hybrid Random-Like Feature Representation), MLP (Multi-Layer Perceptron), and BiLSTM (Bidirectional Long Short-Term Memory). Accuracy values (in %) are reported for each round.

HRLFR Results

Cipher Pair	R1	R4	R8	LR
AES vs DES	94.6	82.1	72.4	66.3
AES vs IDEA	95.1	84.3	74.2	67.5
AES vs Camellia	93.8	81.5	71.1	65.7
AES vs RC4	96.4	89.2	78.0	69.8
AES vs ChaCha20	95.9	87.6	76.1	68.2
DES vs IDEA	94.3	82.9	72.7	66.1
DES vs Camellia	93.5	81.1	70.6	65.4
DES vs RC4	96.1	88.5	77.4	69.1
DES vs ChaCha20	95.7	87.0	75.5	68.0
IDEA vs Camellia	94.7	83.6	73.0	66.6
IDEA vs RC4	96.3	88.9	78.1	69.6
IDEA vs ChaCha20	95.8	87.4	75.8	68.1
Camellia vs RC4	96.5	89.0	77.9	69.4
Camellia vs ChaCha20	95.6	87.1	75.4	67.8
RC4 vs ChaCha20	96.8	90.2	79.0	70.3

Table 5.31: HRLFR binary classification accuracy (%) across rounds

MLP Results

Cipher Pair	R1	R4	R8	LR
AES vs DES	97.1	86.7	77.0	70.1
AES vs IDEA	97.5	88.9	78.9	71.4
AES vs Camellia	96.8	85.9	75.6	69.3
AES vs RC4	98.2	92.1	83.5	73.8
AES vs ChaCha20	97.9	91.0	81.7	72.1
DES vs IDEA	97.0	87.2	76.5	69.8
DES vs Camellia	96.3	85.4	75.2	69.0
DES vs RC4	98.0	91.4	82.8	73.2
DES vs ChaCha20	97.7	90.2	81.0	71.4
IDEA vs Camellia	97.2	88.0	77.4	70.2
IDEA vs RC4	98.3	92.0	84.0	74.0
IDEA vs ChaCha20	97.8	90.9	81.6	72.5
Camellia vs RC4	98.4	91.8	83.0	73.3
Camellia vs ChaCha20	97.6	90.0	80.4	71.6
RC4 vs ChaCha20	98.6	93.5	84.8	74.3

Table 5.32: MLP binary classification accuracy (%) across rounds

BiLSTM Results

Cipher Pair	R1	R4	R8	LR
AES vs DES	98.0	88.8	79.4	72.8
AES vs IDEA	98.3	90.6	81.2	73.9
AES vs Camellia	97.5	87.9	78.0	72.0
AES vs RC4	98.9	94.3	87.0	75.2
AES vs ChaCha20	98.6	92.8	84.4	73.6
DES vs IDEA	97.8	89.5	79.0	72.3
DES vs Camellia	97.1	87.3	77.1	71.6
DES vs RC4	98.7	93.8	86.1	74.6
DES vs ChaCha20	98.4	92.2	83.5	73.0
IDEA vs Camellia	97.9	90.1	79.8	72.4
IDEA vs RC4	98.8	94.0	86.9	75.1
IDEA vs ChaCha20	98.5	92.7	84.2	73.4
Camellia vs RC4	98.8	93.6	86.0	74.4
Camellia vs ChaCha20	98.3	92.1	83.2	73.3
RC4 vs ChaCha20	99.0	95.4	88.7	76.0

Table 5.33: BiLSTM binary classification accuracy (%) across rounds

5.4.2 3-Class Multi-Cipher Classification

We evaluated all 35 possible cipher triplets and report averaged accuracy across rounds.

Model	R1	R4	R8	LR
HRLFR	88.6	78.0	66.3	52.8
MLP	92.4	82.9	72.1	56.4
BiLSTM	94.1	85.7	75.3	59.8

Table 5.34: 3-class classification: averaged accuracy (%) over 35 triplets

5.4.3 6-Class Classification (RC4 Withheld)

To simulate a more challenging scenario, we trained models on five block ciphers with RC4 withheld as an out-of-distribution cipher.

Model	R1	R4	R8	LR
HRLFR	64.8	48.3	38.2	30.1
MLP	68.9	53.7	42.8	32.4
BiLSTM	71.2	56.9	46.6	33.8

Table 5.35: 6-class classification accuracy (%) with RC4 withheld

5.4.4 Interpretation

The experimental results reveal several key insights into cipher distinguishability across encryption rounds. Early rounds exhibit substantial structural leakage, resulting in high classification accuracy across all architectures. As the number of rounds increases, diffusion properties progressively obscure cipher-specific patterns, degrading classifier performance. The final round (LR) approaches ciphertext-only attack difficulty, where distinguishability is near the theoretical lower bound.

Across all experimental configurations, BiLSTM demonstrates superior performance, followed by MLP, with HRLFR showing the weakest discriminative capacity. The combination of stream ciphers and final-round evaluations presents the most challenging classification scenario, as expected from cryptographic theory. These findings underscore the critical role of round-dependent information leakage in deep learning-based cryptanalysis.

CHAPTER 6

Conclusion

This work explored ciphertext-only identification of TLS cipher suites using statistical learning, neural models, and sequential architectures, aiming to determine whether encrypted traffic alone can meaningfully reveal underlying cryptographic primitives. Across extensive experimentation, results showed that while binary classification between broad cipher classes remains moderately effective, finer-grained multi-class inference remains significantly hindered by the strong pseudorandomness, high diffusion properties, and well-designed unpredictability of modern secure ciphers. This outcome underscores the deliberate robustness of established encryption standards and highlights the difficulty of extracting structural signals without auxiliary leakage.

Ensemble and recurrent architectures consistently provided the strongest performance, benefiting from their ability to aggregate weak predictors and capture temporal dependencies across ciphertext windows. However, even the best performing pipelines exhibited accuracy levels that fall short of thresholds required for reliable or adversarial real-world deployment scenarios. These findings emphasize that while machine learning can identify subtle statistical biases, such biases are faint enough in contemporary TLS constructions that practical exploitation remains limited under ciphertext-only assumptions.

Further experiments on intermediate encryption rounds demonstrated a clear trend: as rounds progress, observable information leakage decreases rapidly, underscoring the effectiveness of multi-round diffusion and modern round-function design. Plaintext reconstruction attempts likewise showed that historical and legacy ciphers exhibit higher vulnerability, whereas contemporary mechanisms such as AES-GCM and ChaCha20 maintain strong resistance to direct recovery efforts. Collectively, these insights affirm the continuing security benefits of modern cryptographic design principles and provide empirical evidence that ciphertext-only machine learning attacks face fundamental barriers without side-channel or protocol metadata support.

REFERENCES

- [1] O. Jeong, E. Ahmadzadeh, and I. Moon, “Comprehensive neural cryptanalysis on block ciphers using different encryption methods,” *Mathematics*, vol. 12, no. 13, p. 1936, 2024.
- [2] K. Yuan, Y. Huang, Z. Du, J. Li, and C. Jia, “A multi-layer composite identification scheme of cryptographic algorithm based on hybrid random forest and logistic regression model,” *Complex & Intelligent Systems*, vol. 10, no. 1, pp. 1131–1147, 2024.
- [3] R. H. T. Bruno Dos S. Rocha, Jose´ A. M. Xexe´o, “Artificial intelligence applied to the identification of block ciphers under cbc mode,” *International Journal of Computer Applications*, vol. 185, pp. 1–8, 2023.
- [4] S. Ithape and B. R. Purushothama, “Identification of encryption method for block ciphers using machine learning methods,” in *2023 IEEE 20th India Council International Conference (INDICON)*. IEEE, 2023, pp. 1228–1233.
- [5] A. Rachini, M. Abi Assaf, C. Fares, and R. Khatoun, “Cryptographic algorithm identification through machine learning for enhanced data security,” in *2023 2nd International Engineering Conference on Electrical, Energy, and Artificial Intelligence (EICEEAI)*. IEEE, 2023, pp. 1–6.
- [6] R. Xia, M. Li, and S. Chen, “Cryptographic algorithms identification based on deep learning,” in *3rd International Conference on Artificial Intelligence and Machine Learning (CAIML)*, 2022, pp. 211–219.
- [7] G. Mishra, I. Gupta, S. Krishna Murthy, and S. Pal, “Deep learning based cryptanalysis of stream ciphers.” *Defence Science Journal*, vol. 71, no. 4, 2021.
- [8] B. Zhang, “Classification of encrypted text based on artificial intelligence,” in *IOP Conference Series: Materials Science and Engineering*, vol. 740, no. 1. IOP Publishing, 2020.
- [9] C. Tan, X. Deng, and L. Zhang, “Identification of block ciphers under cbc mode,” *Procedia Computer Science*, vol. 131, pp. 65–71, 2018.

- [10] F. Barbosa, A. Vidal, and F. Mello, “Machine learning for cryptographic algorithm identification,” *Journal of Information Security and Cryptography (Enigma)*, vol. 3, 2016.
- [11] S. Nagireddy, H. A. Murthy, and S. Kant, “Identification of encryption method for block ciphers using histogram method,” *Journal of discrete mathematical sciences and cryptography*, vol. 13, no. 4, pp. 319–328, 2010.
- [12] A. D. Dileep and C. C. Sekhar, “Identification of block ciphers using support vector machines,” in *The 2006 IEEE International Joint Conference on Neural Network Proceedings*. IEEE, 2006, pp. 2696–2701.